



THE HONG KONG POLYTECHNIC UNIVERSITY

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EIE1005 Fundamental AI and Data Analytics

Lecture 2: AI and Data Analytics for Computer Vision

Lecturer: Prof. Y.L.Chan
Room: DE606
Tel: 27666213
Email: enylchan@polyu.edu.hk

Schedule

(Monday Lecture)

- Lecture @Z209
 - Week 1, 2, 7, 10
 - Monday 16:30 – 18:20
 - Week 5 – Guest Lecture
 - NVIDIA Seminar (Attendance Required)
- Workshop
 - Lab001 Wednesday 12:30 – 14:20
 - Lab002 Wednesday 14:30 – 16:20
 - Lab003 Tuesday 11:30 – 13:20
 - Wk 3 and 4 @U204a/b/c
 - Wk 8, 9, 11, and 12 @CF105
 - Wk 5 Optional for Mini-Project (CF105)
- Test 1 and 2 @Z209
 - Week 6 – Test 1
 - Week 13 – Test 2
 - Monday 16:30 – 18:20

Month	Week	Mon	Tue	Wed	Thurs	Fri	Sat	Sun	Sem. Week
Aug 2026	1	31	1	2	3	4	5	6	1
Sept	2	7	8	9	10	11	12	13	2
	3	14	15	16	17	18	19	20	3
	4	21	22	23	24	25	26	27	4
	5	28	29	30	1	2	3	4	5
Oct	6	5	6	7	8	9	10	11	6
	7	12	13	14	15	16	17	18	7
	8	19	20	21	22	23	24	25	8
	9	26	27	28	29	30	31	1	9
Nov	10	2	3	4	5	6	7	8	10
	11	9	10	11	12	13	14	15	11
	12	16	17	18	19	20	21	22	12
	13	23	24	25	26	27	28	29	13
Dec	14	30	1	2	3	4	5	6	Exam.
	15	7	8	9	10	11	12	13	
	16	14	15	16	17	18	19	20	

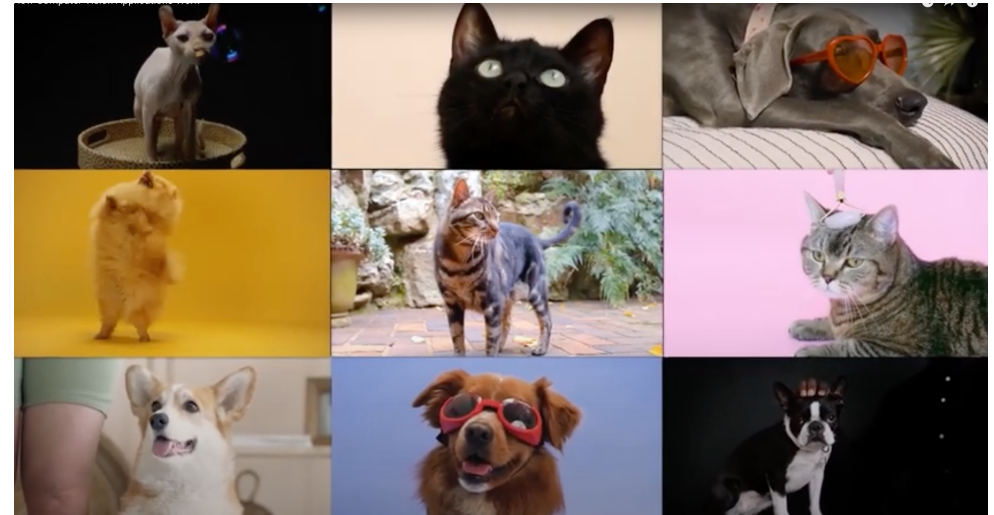
Submission

- Group project

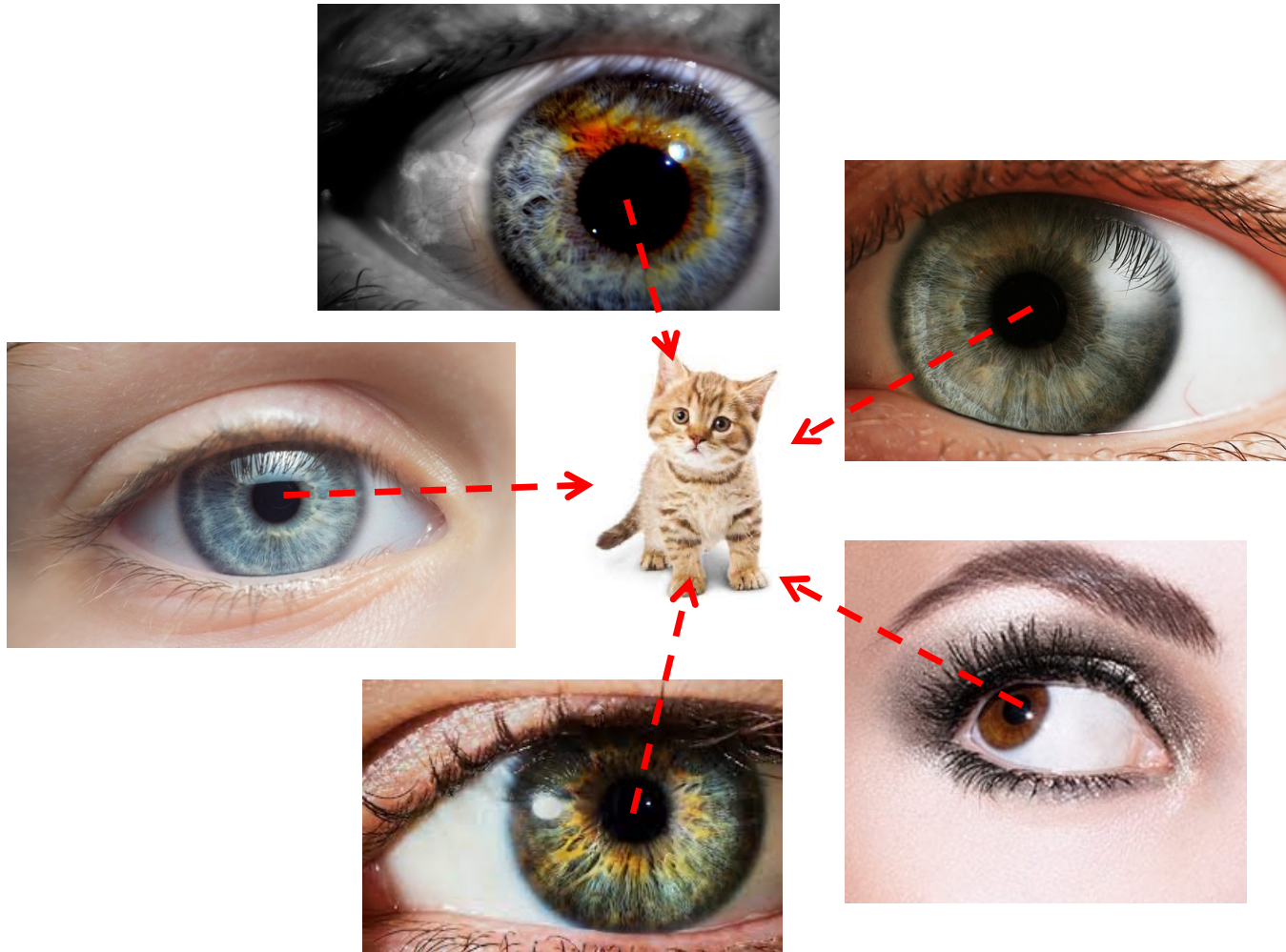
- This group project and demonstration provides students the opportunity to get familiar with the training process of using neural networks to a computer vision problem. The open-ended task is to use Teachable Machine (<https://teachablemachine.withgoogle.com/>) to train an AI model to recognize objects that may be applied to their disciplines. It aims to encourage self-learning, to train logical thinking, and to practice reporting/demonstration skill.
- **Preparation: Mini-project workshop, briefing videos.**
- **Form group made up of five to six students** who will work together to select a topic and develop an AI model to recognize objects.
- **Demonstration: create a demonstration video in 5-7 minutes.**
- **Deadlines and important dates:**
 - Submission of the group composition via Blackboard by 30 Sept., 2026
 - Submission of the video demonstration via Blackboard by 28 Nov., 2026



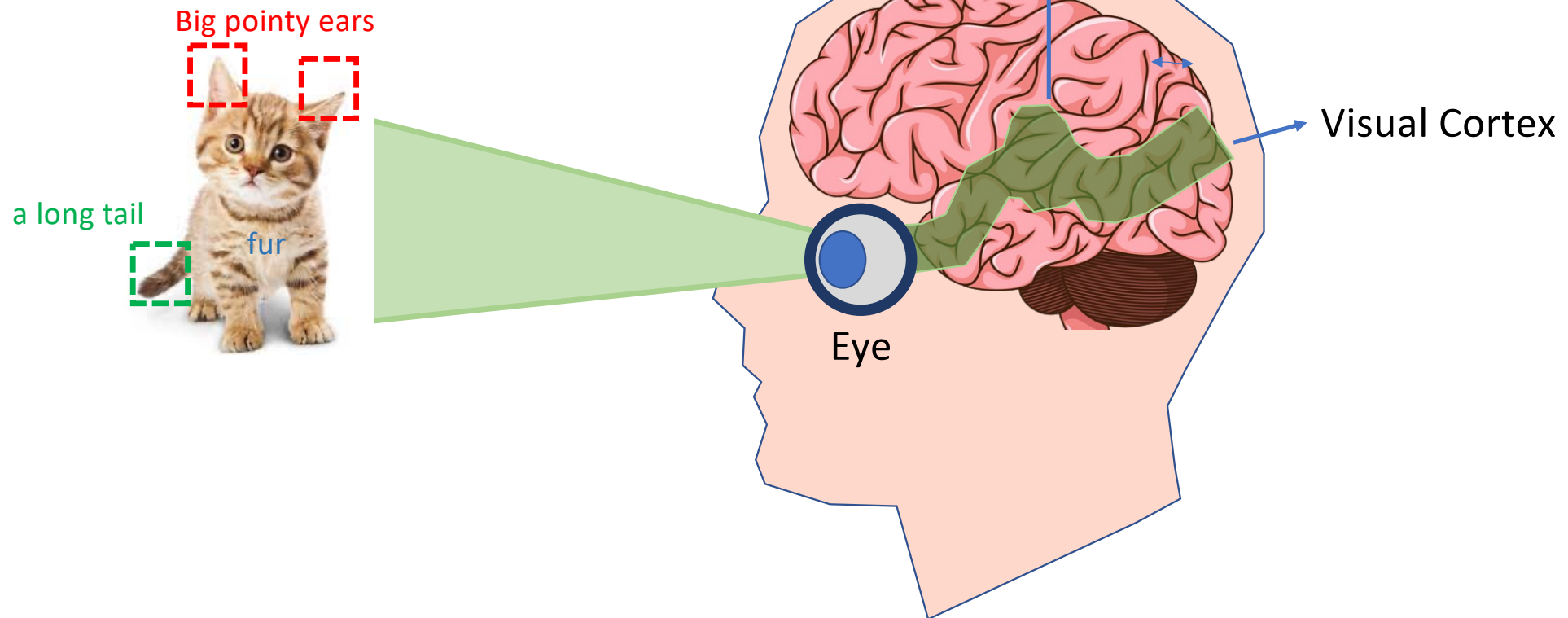
- Sometimes children **mix up cats and dogs** especially if they were not exposed to different breeds or colors or sizes of animals before.



As soon as they have seen enough cats and dogs and other furry animals they have learned the difference.



Pattern recognition (PR): we rely on patterns or features to figure out what type of objects.



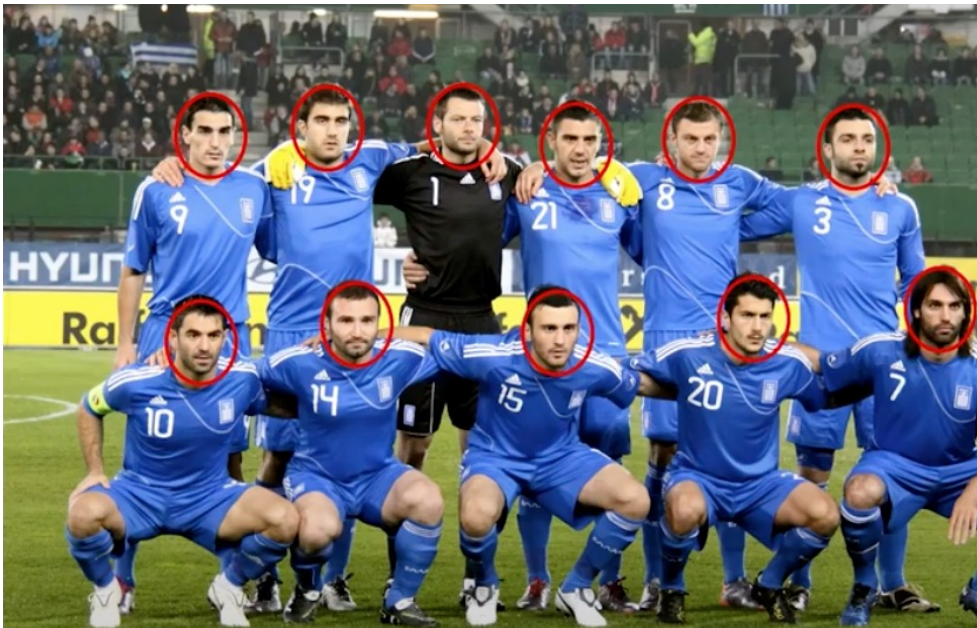
How
COMPUTER VISION
Application works

Computer Vision

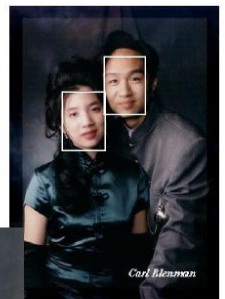
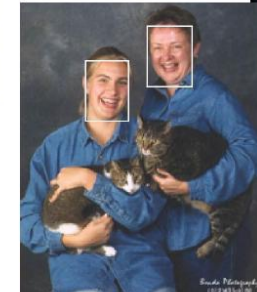
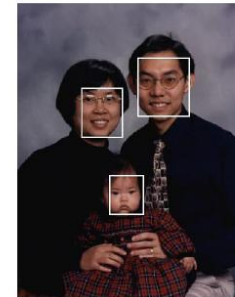
- “Computer Vision” is an area of Machine Learning that deals with image recognition and classification.
- Computer Vision models can be developed to accomplish tasks
 - like facial recognition, identifying which breed a dog belongs to and even identifying a tumor from CT scans,

Possibilities are endless.

Face Detection and Recognition



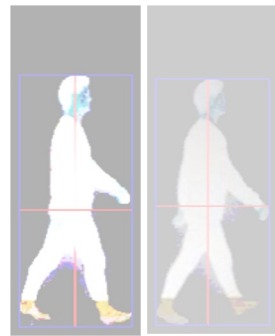
Face Detection



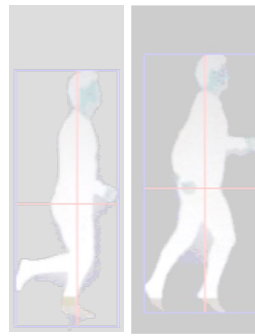
Face Detection for Privacy Protection



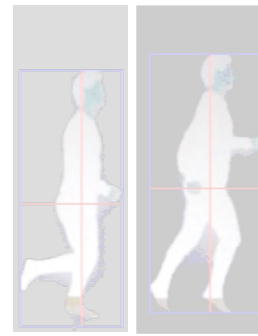
Human Action/Activity Recognition



Step



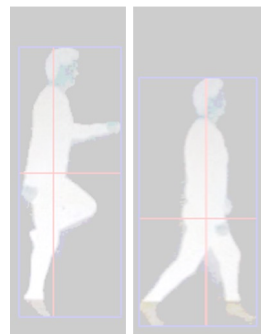
Run



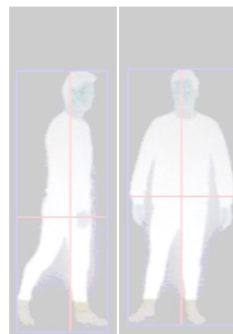
Hop



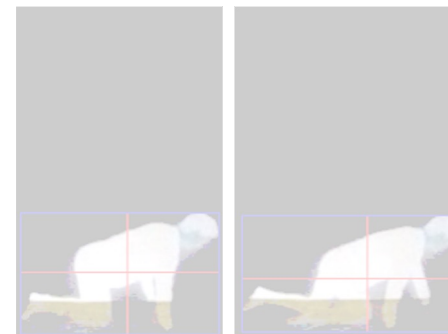
Jump



Skip



Swivel turn



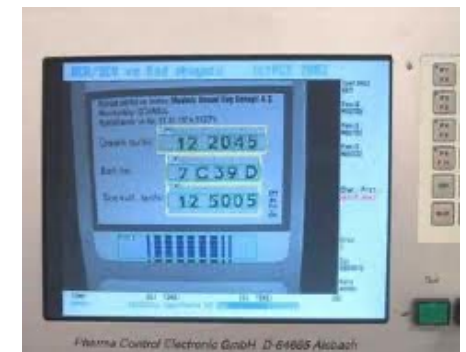
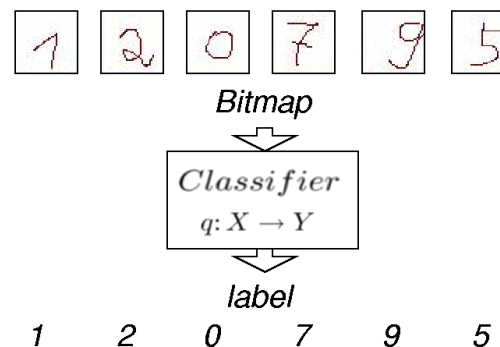
Crawl

Optical Character Recognition (OCR)

- Printing
- Handwritten

License plate Recognition (LPR) is the capacity to capture photographic video or images from license plates and transform the optical data into digital information in real-time.

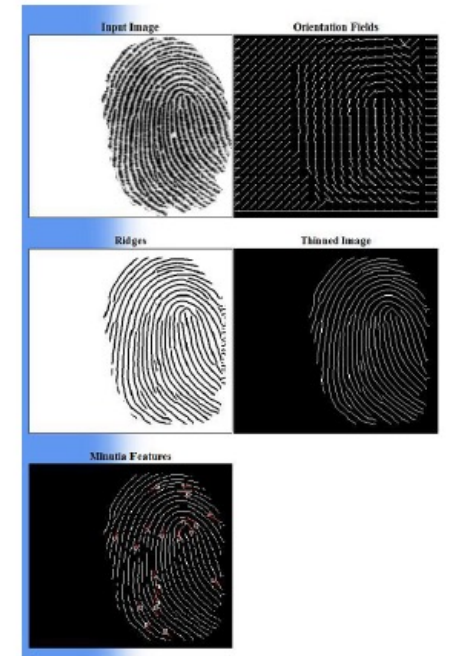
widely used technology for vehicle management operations such as Ticketless Parking (off-street and on-street), Tolling, stolen vehicles detection, smart billing,



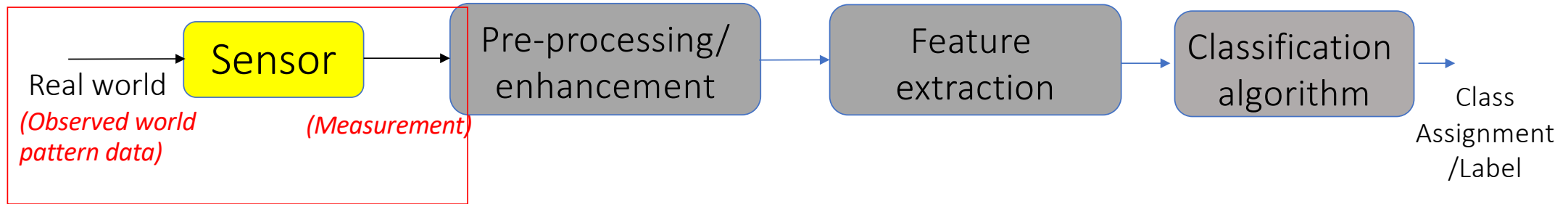
Identification and Authentication



Minutia Extraction:
Determine location
and orientation of
ridge bifurcations
and ridge
terminations

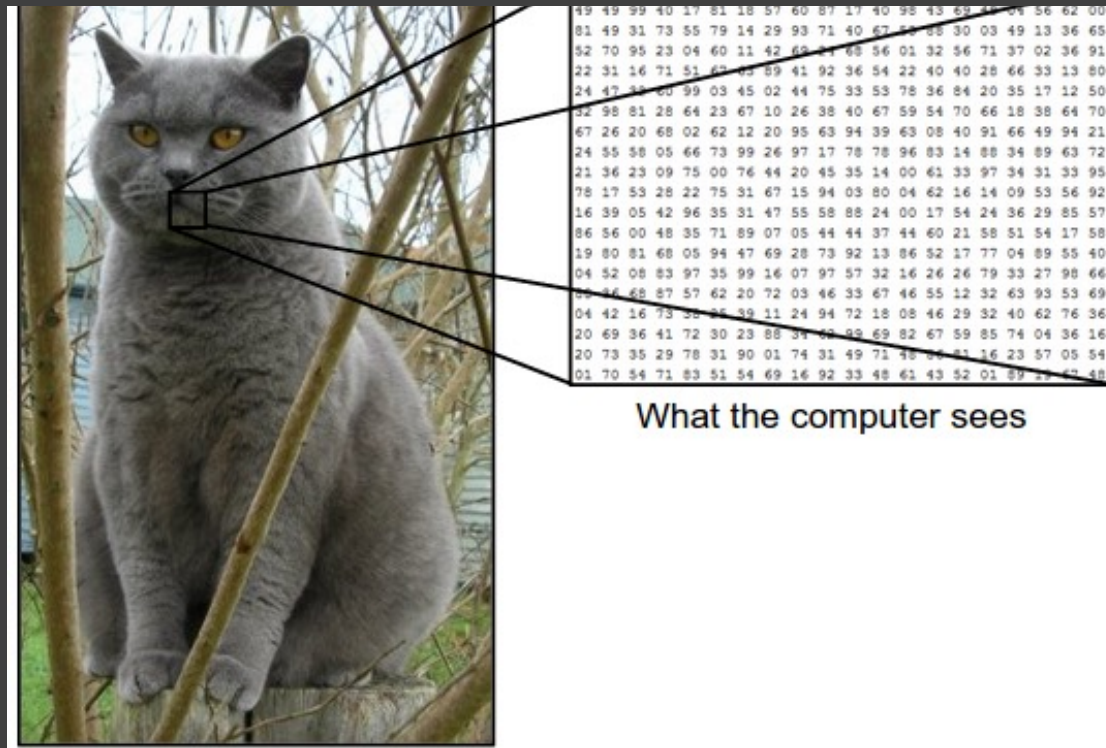


Components of an Image Recognition and Classification/Pattern Recognition (PR) system



- PR system needs some input from the real world that it perceives with sensors.
- Such a system can work with any type of data: **images, videos, numbers, or texts.**

How do computers “see” images?

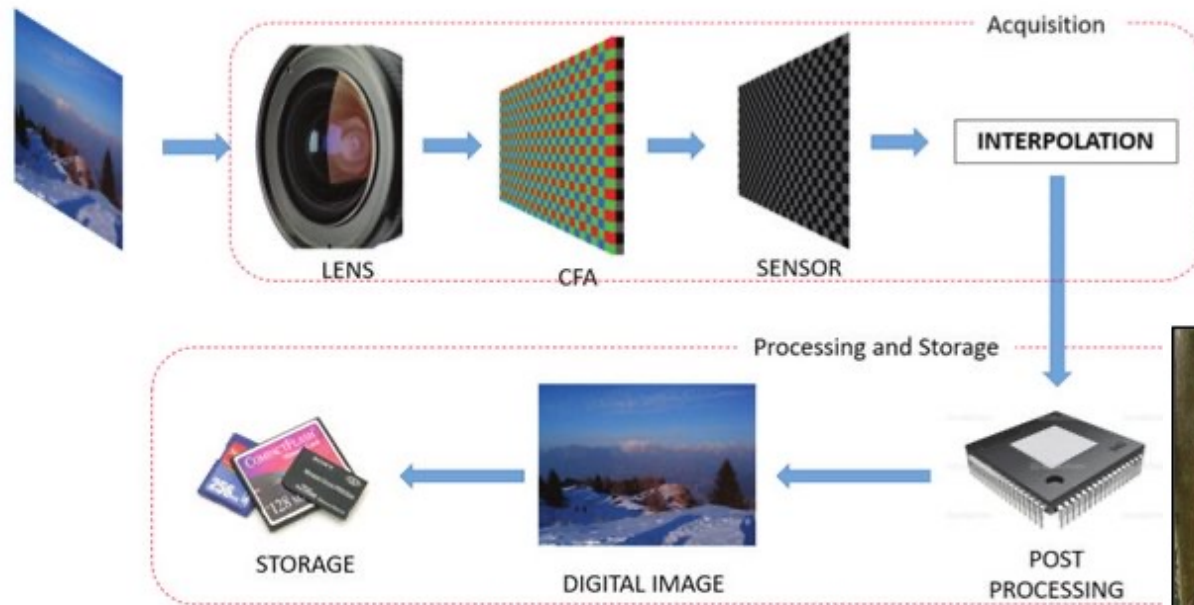


- Computers don't see images the same way humans do
- They can **only understand numbers.**
- So the first step in any computer vision problem is **to convert the information that an image contains to a machine-readable form.**

How do computers “see” images?

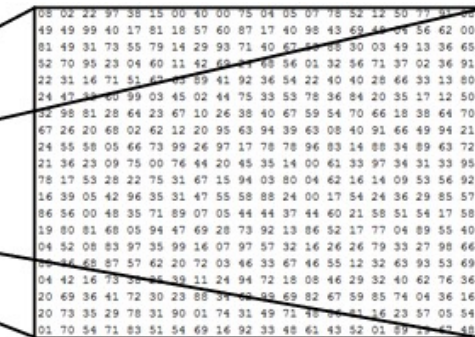
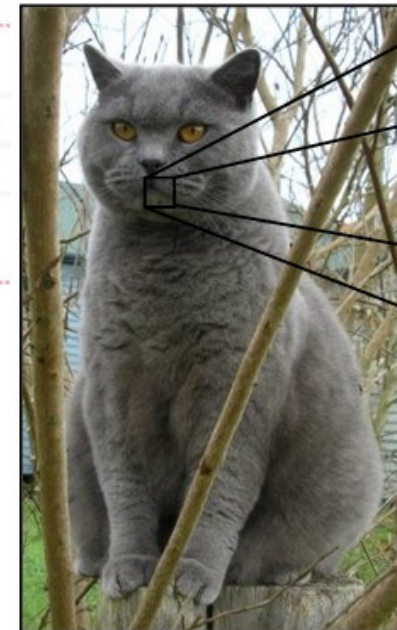
Camera can capture and convert photos into digital form.

Understanding what's in the photo is much more difficult.



Closely mimic how the human eye can capture light and color.

easy task

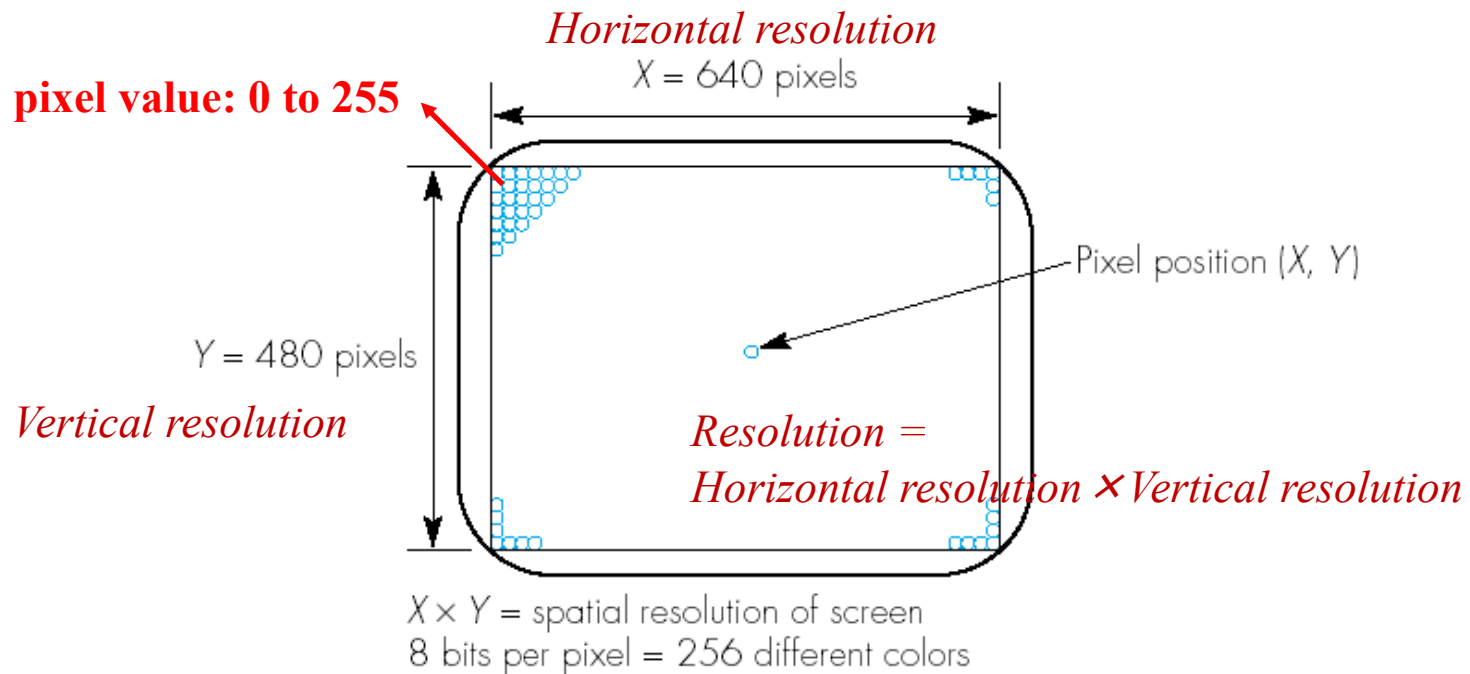


What the computer sees

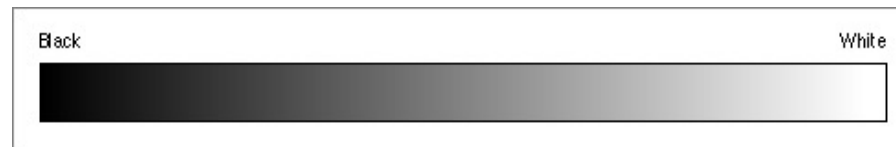
image classification → 82% cat
15% dog
2% hat
1% mug

Digital Image Basics

- All images are displayed in the form of a two dimensional matrix of individual picture elements called **pixels**.

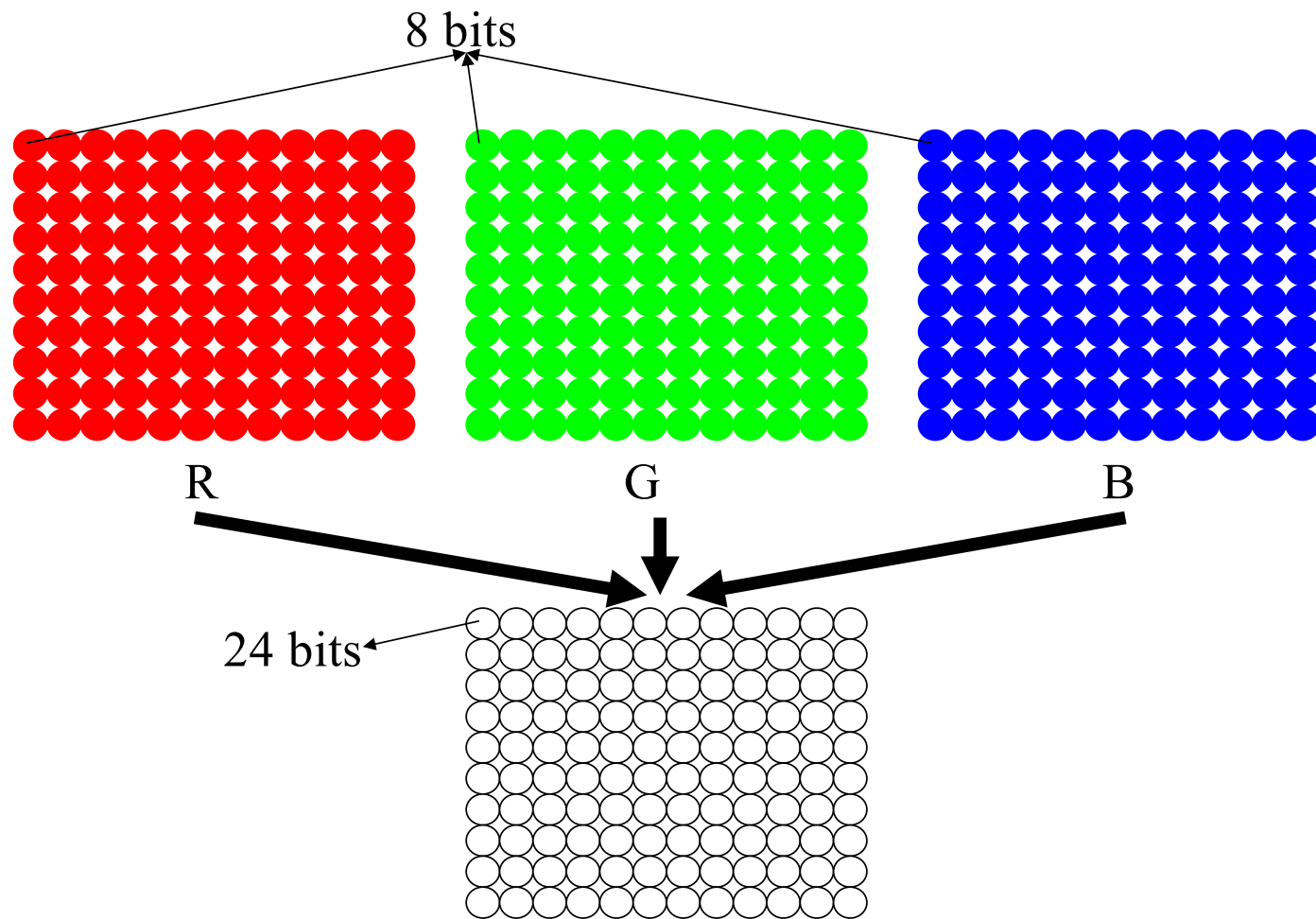


- Grayscale Image: Good quality black-and-white pictures can be obtained by using 8 bits per picture element (pixel)
 - 256 different levels of gray per element.



- **Color Image**: A whole spectrum of colors can be produced by using different proportions of the 3 primary colors - red, green and blue.

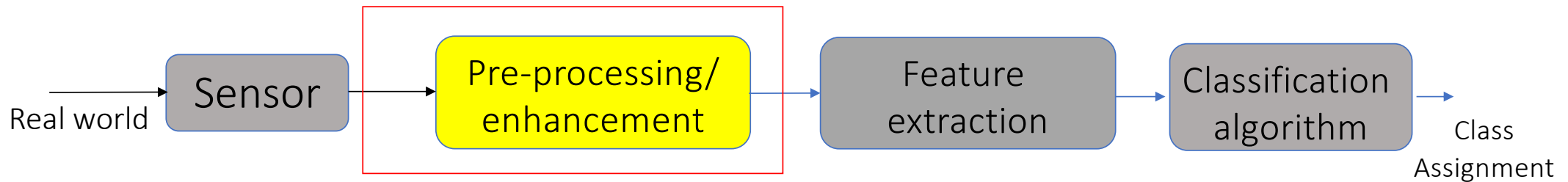




Exercise

- Derive the size of the image with RGB digitization format. Assume that each has the resolution of 1920×1080 and the pixel of each component is represented by 8 bits

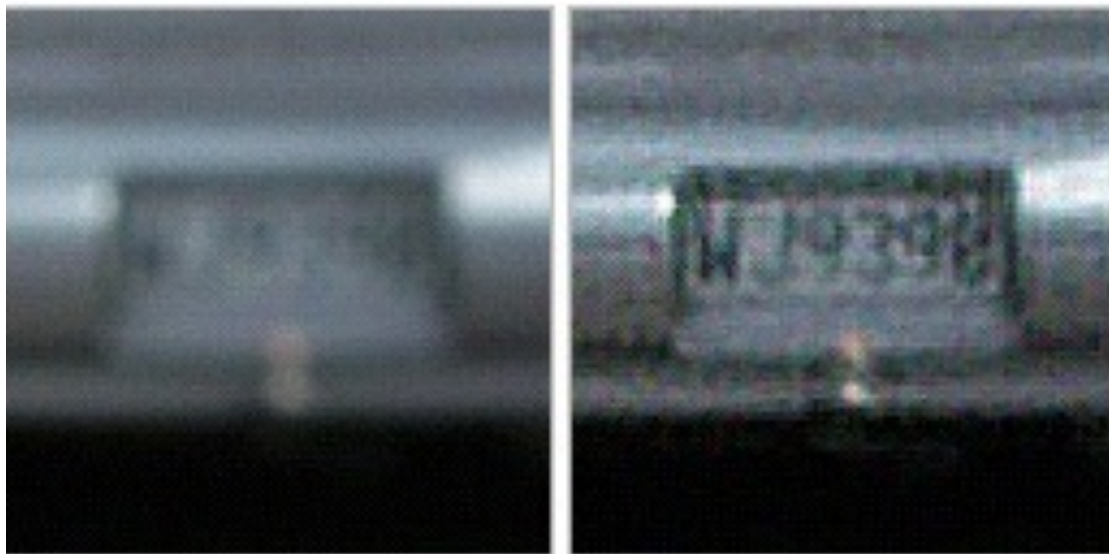




- Having received some information as the input, the algorithm performs **pre-processing and enhancement**.
 - segmenting something interesting from the background. e.g: given a group photo and a familiar face attracts your attention.



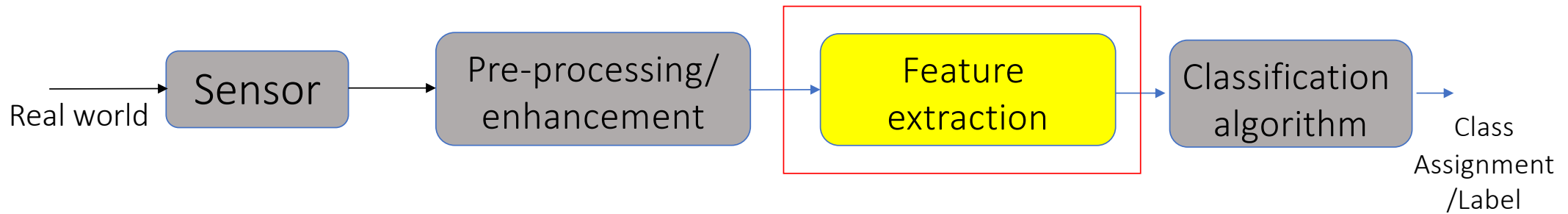
Image Enhancement



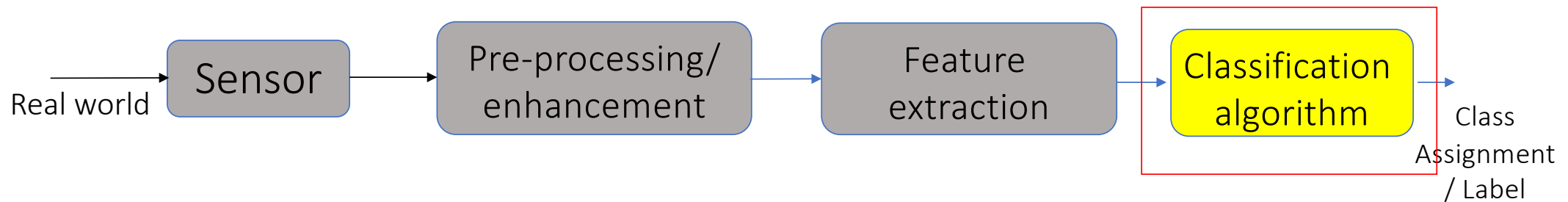
Before

After

- Sharpening image features such as edges, boundaries, or contrast to make an image easier for analysis/feature extraction.
- The enhancement does not increase the inherent information content of the data, but it **increases the dynamic range of the chosen features** so that they can be detected easily.



A **feature extractor** is a program that inputs the data (image) and extracts features that can be used in classification/clustering.



- For the machine to search for patterns in data, it should be pre-processed and converted into a form (features) that a computer can understand.
- **Classification** algorithms:
 - depending on the information available about the problem, can be used to get valuable results.
 - In classification, the algorithm assigns labels to data based on the predefined features.

Classification in PR system

- A **class** is a set of objects having some important properties in common.
- A **feature extractor** is a program that inputs the image and extracts features that can be used in classification.
- A **classifier** is a program that inputs the feature vector and assigns it to one of a set of designated classes or to the “reject” class.



Feature Selection and Extraction

- It is important to choose and to extract features that
 1. are **computationally feasible**
 2. lead to “good” PR system
 3. reduce the data to a manageable scale without losing valuable information
- *Feature selection* is the process of choosing input to the classifier and involves judgement.
 - Extracted features should be relevant to the PR task at hand.

Feature Selection

- Discriminating between Apples and Apricots:



$x_1 = \text{radius}$

$x_2 = R+G+B \text{ (color)}$



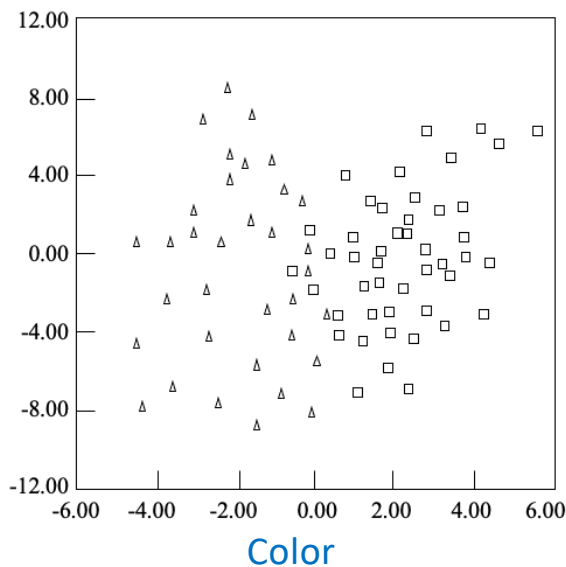
Which feature is more discriminative?

Possible features for Character Recognition

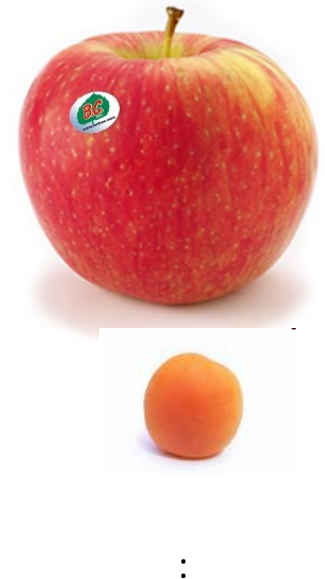
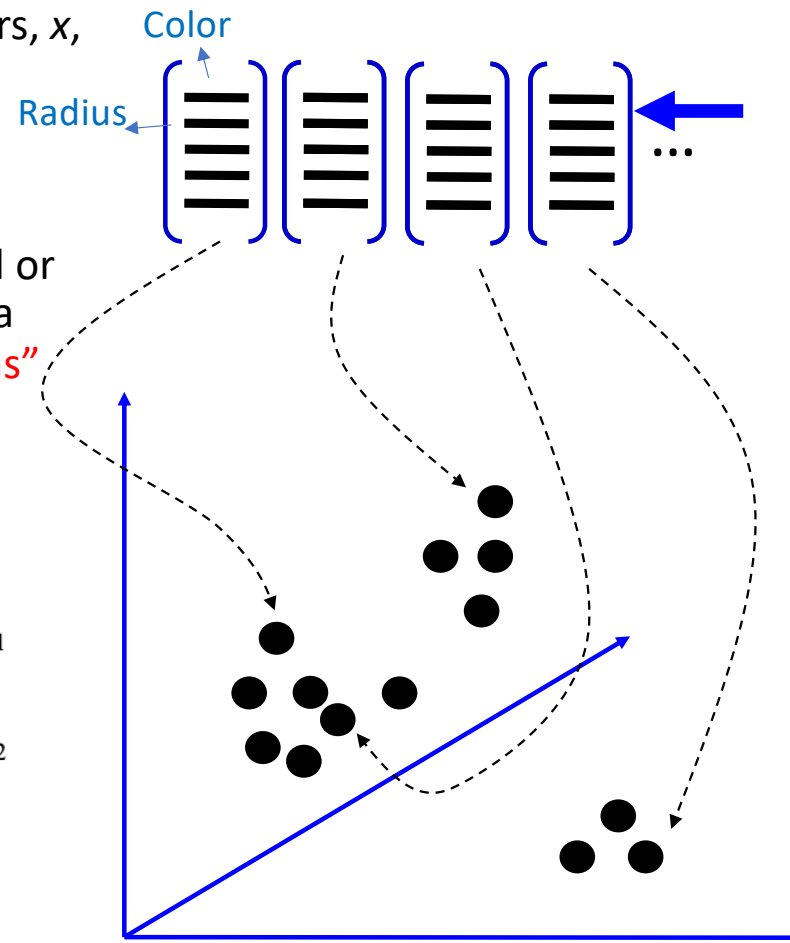
(class) character	area	height	width	number #holes	number #strokes	(cx,cy) center	...	...
'A'	medium	high	3/4	1	3	1/2,2/3	...	...
'B'	medium	high	3/4	2	1	1/3,1/2	...	...
'8'	medium	high	2/3	2	0	1/2,1/2	...	...
'0'	medium	high	2/3	1	0	1/2,1/2	...	...
'1'	low	high	1/4	0	1	1/2,1/2	...	...
'W'	high	high	1	0	4	1/2,2/3	...	...
'l'	high	high	3/4	0	2	1/2,1/2	...	...
'*'	medium	low	1/2	0	0	1/2,1/2	...	...
'_'	low	low	2/3	0	1	1/2,1/2	...	...
'/'	low	high	2/3	0	1	1/2,1/2	...	...

Feature Space – Scatter plots

- **Scatter plots** are plots of sample feature vectors, x , in feature space.
- Excellent visualization tools for determining feature vector distribution in R^n , where $n \leq 3$.
- Scatter plots often facilitate identifying natural or obvious clustering of class-specific feature data and the partitioning of R^n into “**decision regions**” for classification.

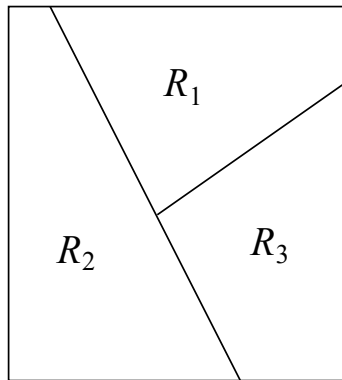


$\triangle$ class w_1
 $\square$ class w_2

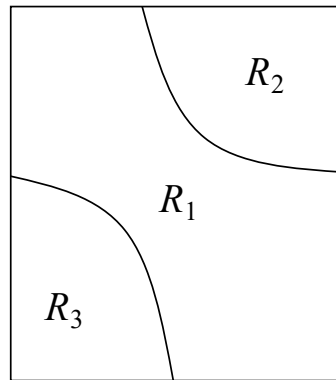


Decision Regions & Boundaries

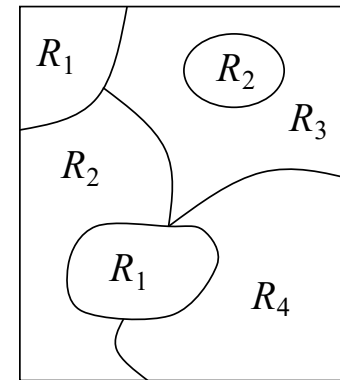
- A *classifier* partitions feature space into class-labeled *decision regions*.
- The border of each decision region is a *decision boundary*.
- PR system is looking for these decision boundaries.



Linear (piecewise)

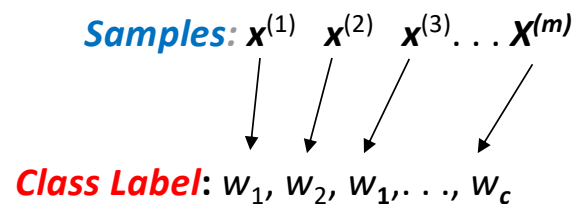


Quadratic (hyperbolic) (Relatively) general



Training Set

- *Training set*, **H** – a set of “typical” patterns, where typical attributes/features or the class or structure of each is known
⇒ provide significant information on how to associate input data with output decision.
- *H* contains many samples/observations of input/output pair:



Number of samples: m

Number of classes: c

Given a collection of records (*training set*)
Each record contains a set of *features*, and the *class*.

Given a collection of records (*training set*)
Each record contains a set of *features*, and the *class*.

Instances (samples, observations)

	$x^{(1)}$	$x^{(2)}$	...	$x^{(50)}$	...	$x^{(m)}$
$x_1^{(j)}$	$x_1^{(1)}$	$x_1^{(2)}$	...	$x_1^{(50)}$	...	$x_1^{(m)}$
$x_2^{(j)}$	$x_2^{(1)}$	$x_2^{(2)}$	...	$x_2^{(50)}$	...	$x_2^{(m)}$
:	:	:	:	:	:	:
$x_{n-1}^{(j)}$	$x_{n-1}^{(1)}$	$x_{n-1}^{(2)}$	...	$x_{n-1}^{(50)}$	...	$x_{n-1}^{(m)}$
$x_n^{(j)}$	$x_n^{(1)}$	$x_n^{(2)}$	...	$x_n^{(50)}$	...	$x_n^{(m)}$
Class	w_1	w_3	...	w_1	...	w_2

Classification: Definition

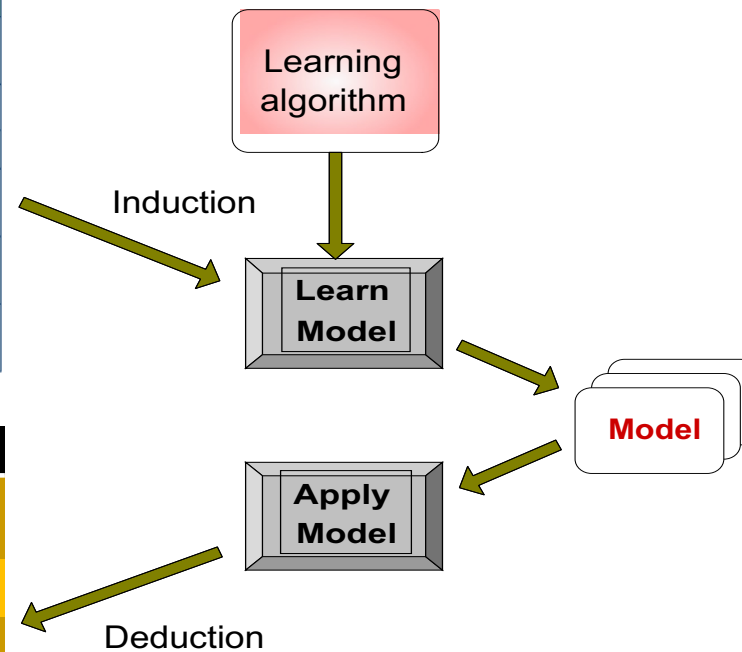
- Given a collection of records (*training set*)
 - Each record contains a set of *features*, and the *class*.
- Find a *model* for the class as a function of the values of other features.
- Goal: previously unseen records should be assigned a class as accurately as possible.
 - A *test set* is used to determine the accuracy of the model. Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to validate it.

Illustrating Classification Task

Training set

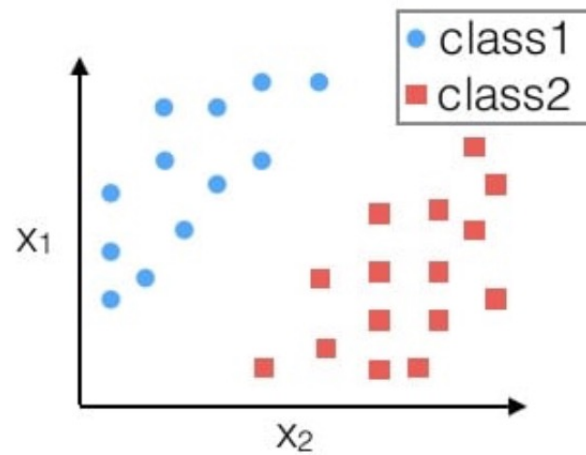
	$x^{(1)}$	$x^{(2)}$	...	$x^{(50)}$	...	$x^{(m)}$
$x_1^{(j)}$	$x_1^{(1)}$	$x_1^{(2)}$	...	$x_1^{(50)}$	...	$x_1^{(m)}$
$x_2^{(j)}$	$x_2^{(1)}$	$x_2^{(2)}$	...	$x_2^{(50)}$	...	$x_2^{(m)}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$x_{n-1}^{(j)}$	$x_{n-1}^{(1)}$	$x_{n-1}^{(2)}$	...	$x_{n-1}^{(50)}$	...	$x_{n-1}^{(m)}$
$x_n^{(j)}$	$x_n^{(1)}$	$x_n^{(2)}$	...	$x_n^{(50)}$	...	$x_n^{(m)}$
Class	w_1	w_3	...	w_I	...	w_2

	$x^{(m+1)}$	$x^{(m+2)}$	...
$x_1^{(j)}$	$x_1^{(1)}$	$x_1^{(2)}$	...
$x_2^{(j)}$	$x_2^{(1)}$	$x_2^{(2)}$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$x_n^{(j)}$	$x_n^{(1)}$	$x_n^{(2)}$	...
Class	?	?	...

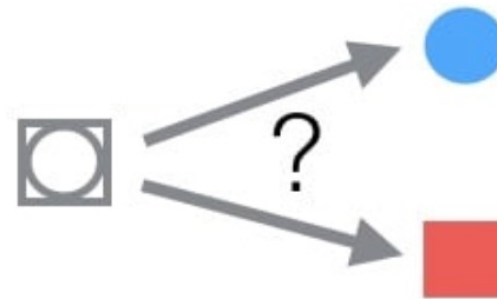


Classification

1. Learn from training data



2. Map unseen (new) data



Example: Image Classification



(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



cat

Image Classification: Problem

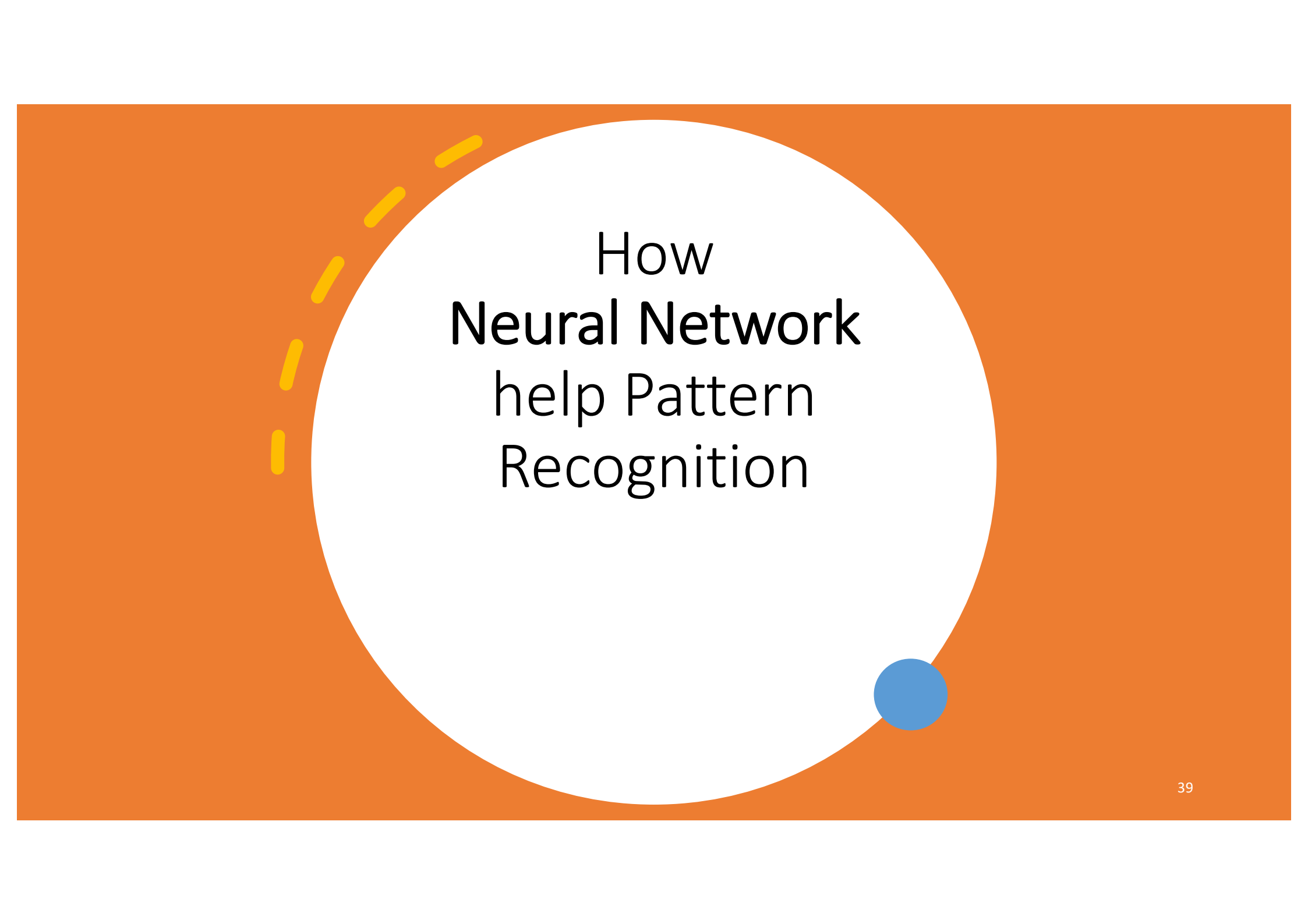


08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	82
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	45	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	55	88	30	03	49	13	36	65
52	70	95	23	04	60	11	42	69	24	68	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	83	59	41	92	36	54	22	40	40	28	66	33	13	80
24	47	32	60	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
52	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
85	46	68	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	69
04	42	16	73	32	55	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	62	89	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	86	81	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	19	62	48

What the computer sees

image classification

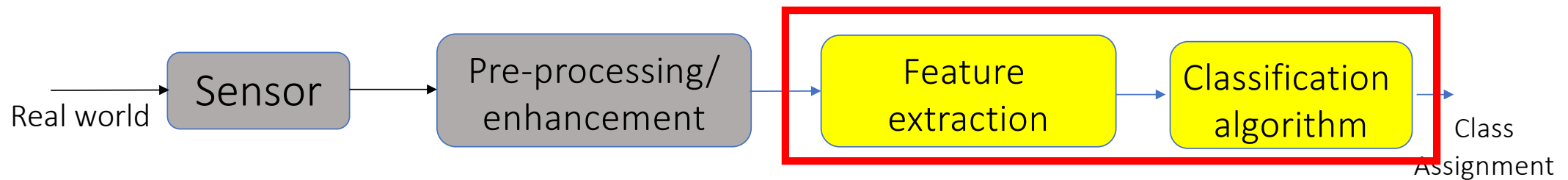
82% cat
15% dog
2% hat
1% mug



How Neural Network help Pattern Recognition

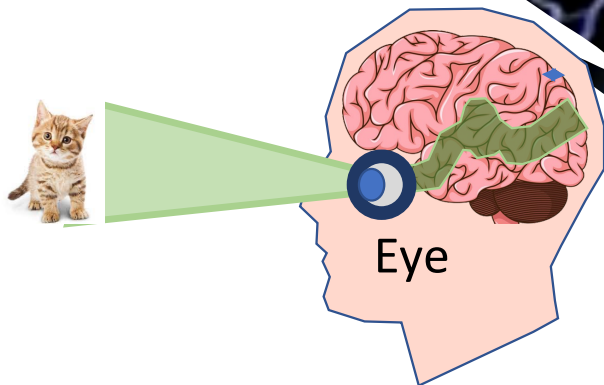
Feature Selection

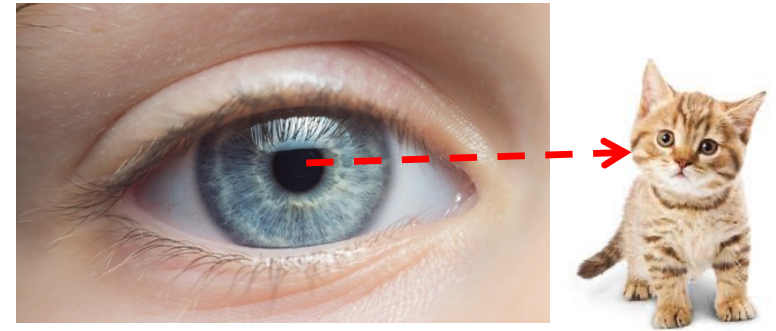
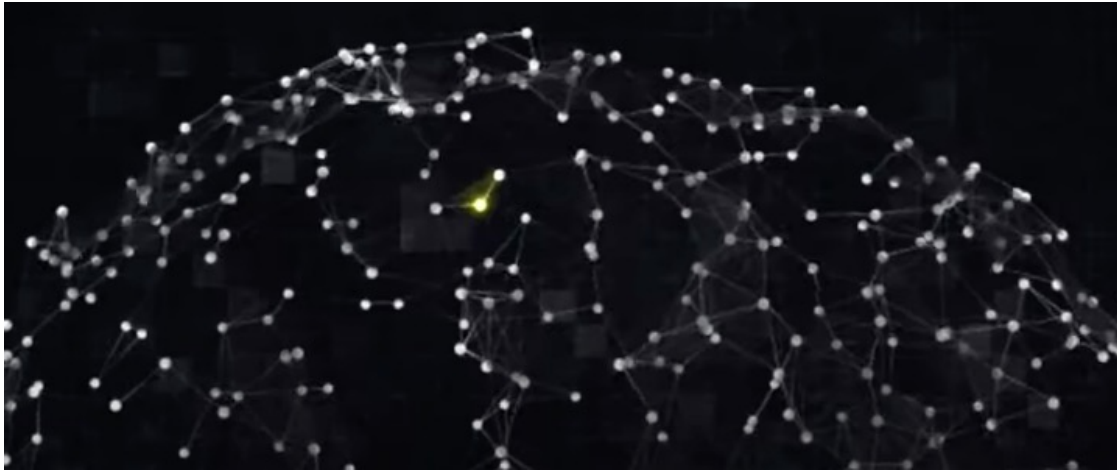
- Discriminating between Apples, Apricots, Oranges:



The Brain

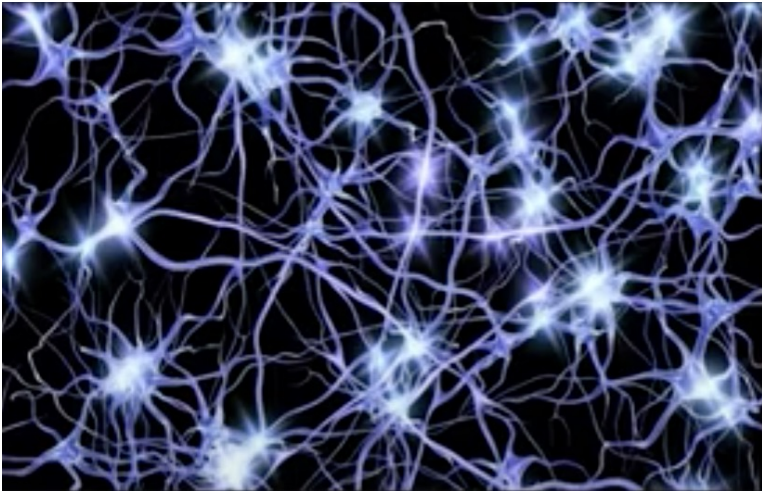
- 2% of the weight of a person, but consumes about 20% of the energy.
- This energy is being used to process and transmit all kinds of signals, in particular, visual information.
- The brain is nothing but a network of neurons: very complex network that is able to allow us to do all the sophisticated perception tasks



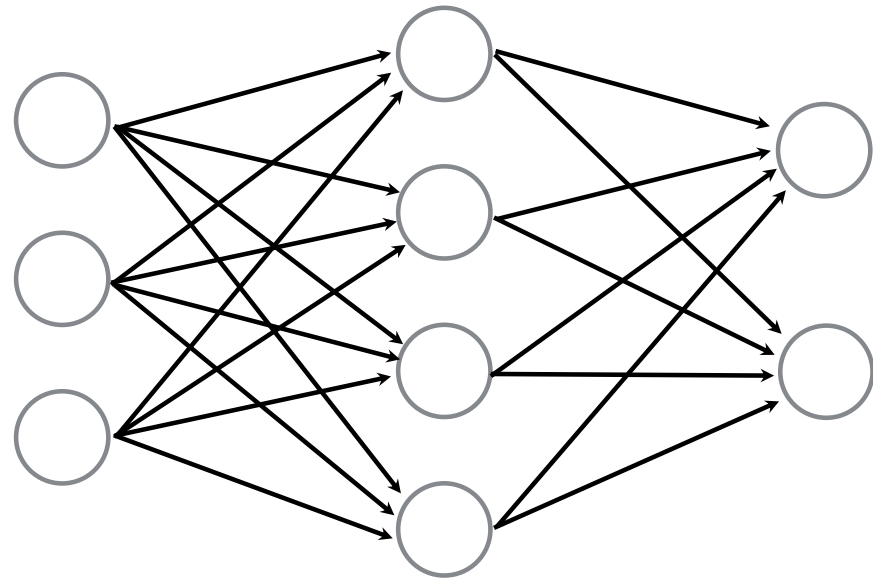


Whenever we see a new type of cat, the same connection strengthens making it easier for us to recognize an animal as a cat next time

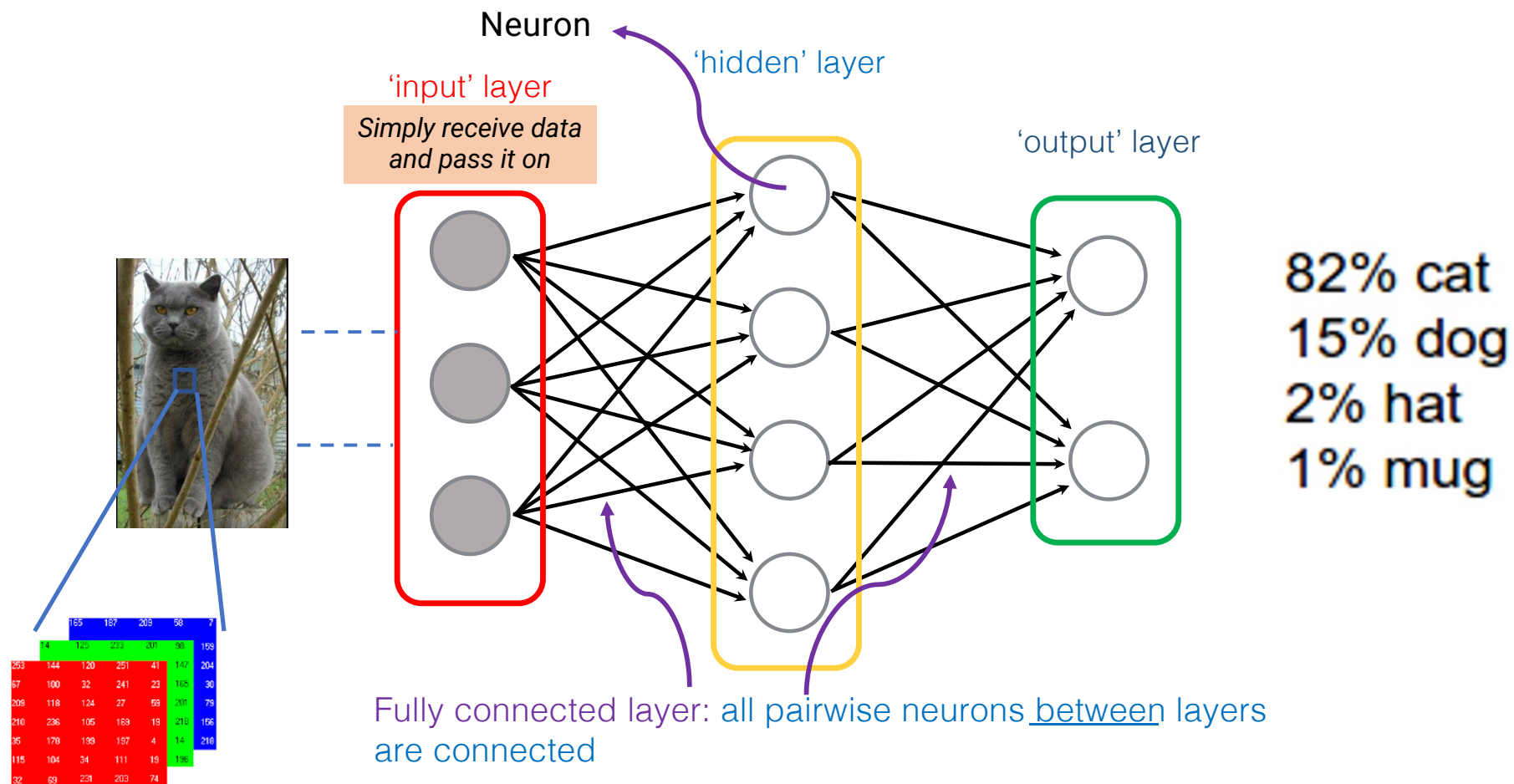
NEURAL NETWORK



ARTIFICIAL NEURAL NETWORK



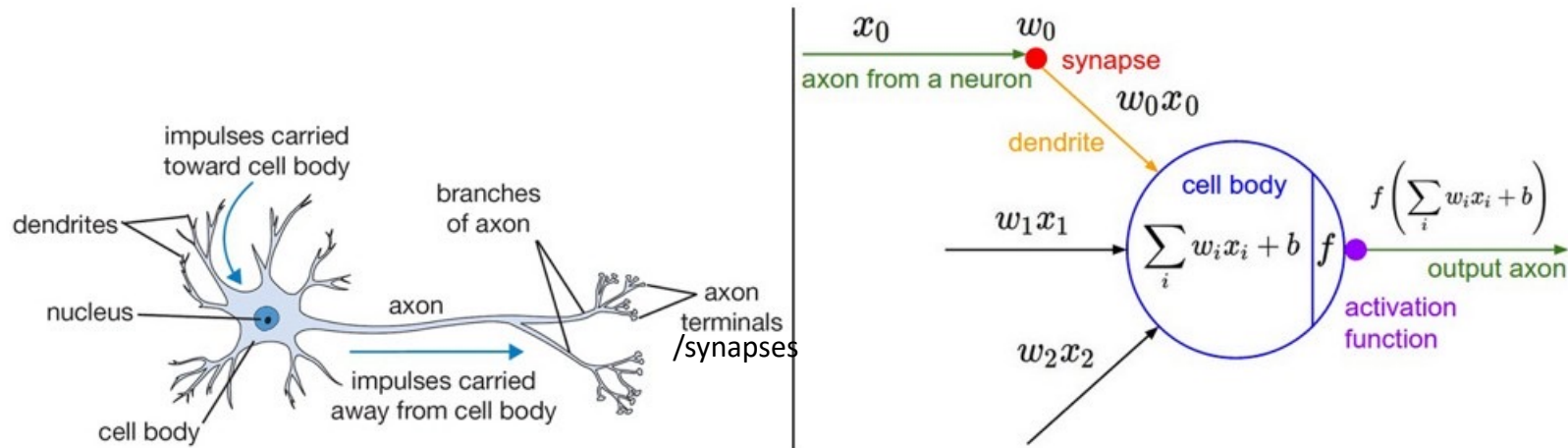
Disclaimer: we are using simplified explanations and formulation to make concepts more digestible



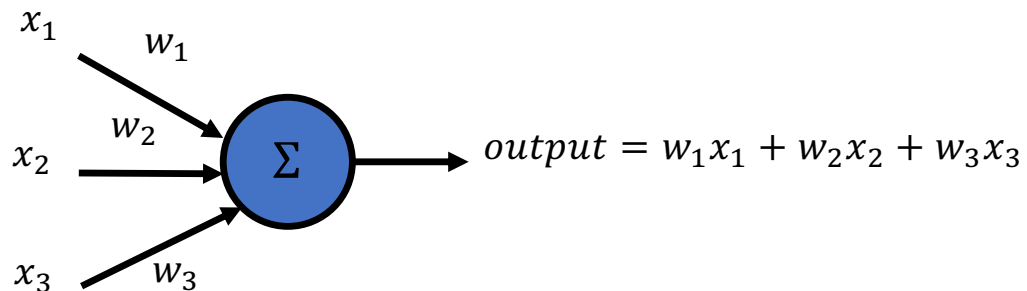
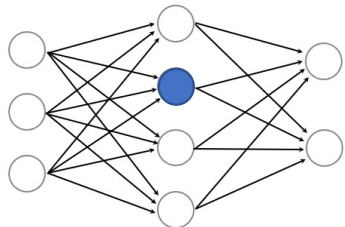
- Neural networks are **loosely** inspired by biology.
- But they certainly are **not** a model of how the brain works, or even how neurons work.

Neuron

The **dendrites** actually receive signals from, for instance, other neurons. And then the signals come in. It's a very simple computation to produce an output. The output then travels through what's called the **axon**, and goes to the other end and goes to what are called **synapses**, which is where one neuron makes a connection with another neuron.



A cartoon drawing of a biological neuron (left) and its mathematical model (right).

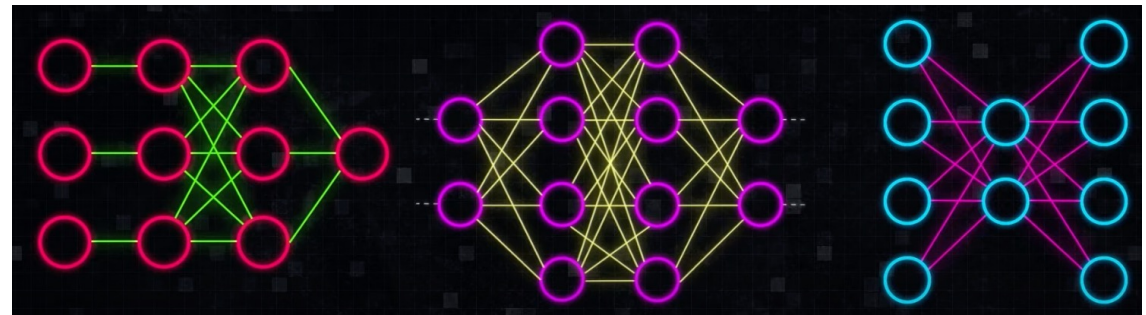
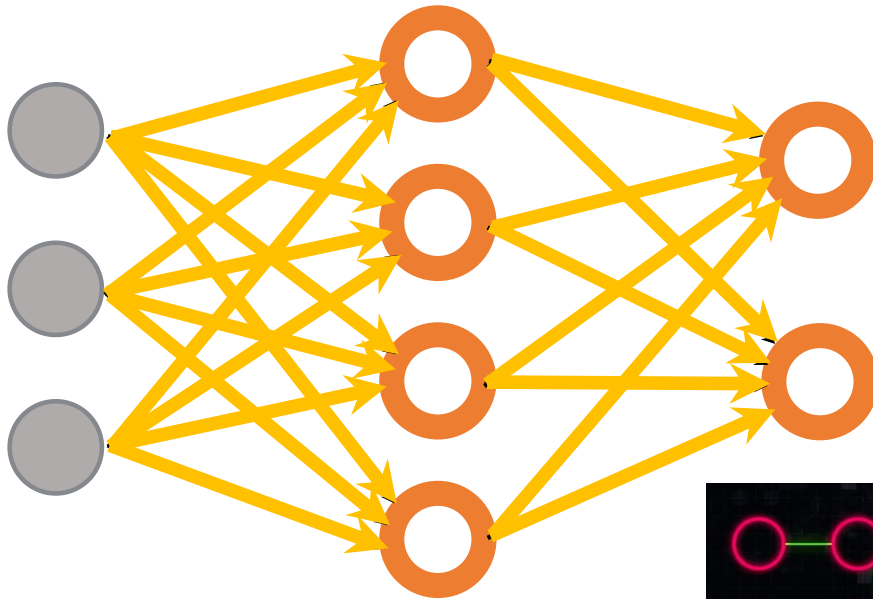


How many neurons (perceptrons)?

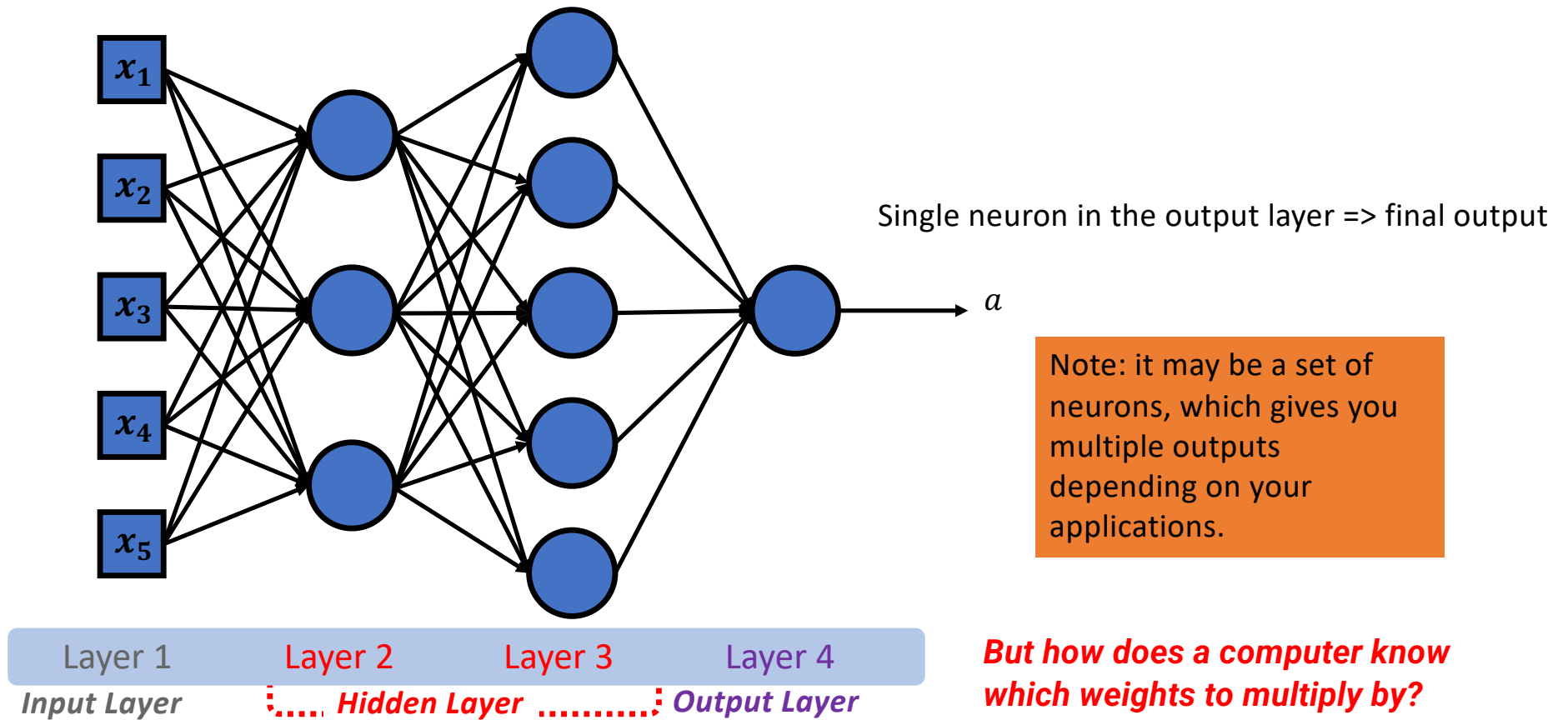
$$4 + 2 = 6$$

How many weights (edges)?

$$(3 \times 4) + (4 \times 2) = 20$$



A Neural Network with multiple hidden layers



Case 1: Learning/Training



Cat



Dog

Learning based on the obvious feature – Color
White Pet → Cat
Black Pet → Dog

Classification Label



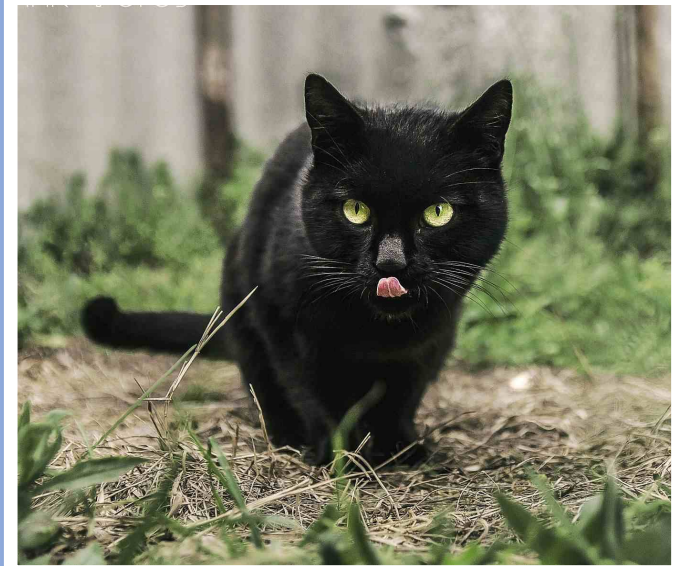
Know nothing about animals
Start to recognize “Cat” and “Dog”

Case 2: Testing

Learned feature – based on Color
White Pet → Cat
Black Pet → Dog

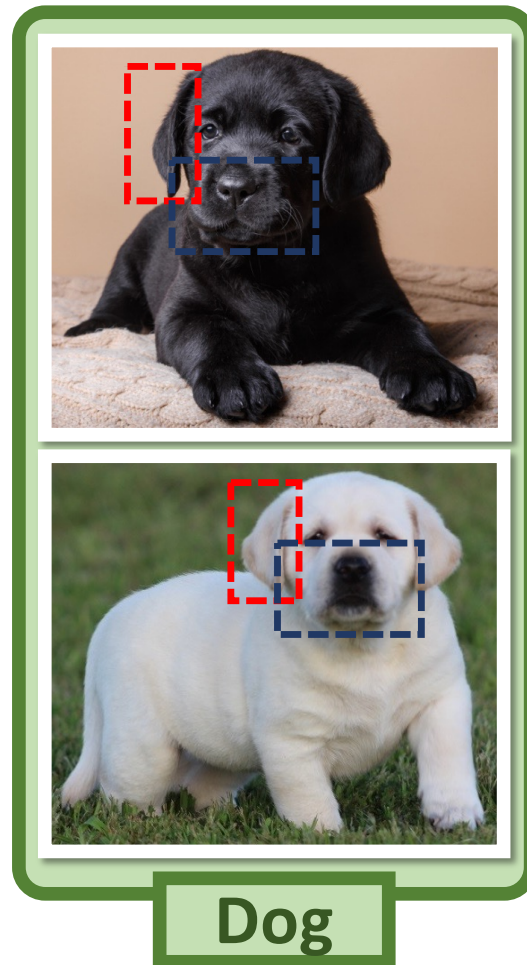
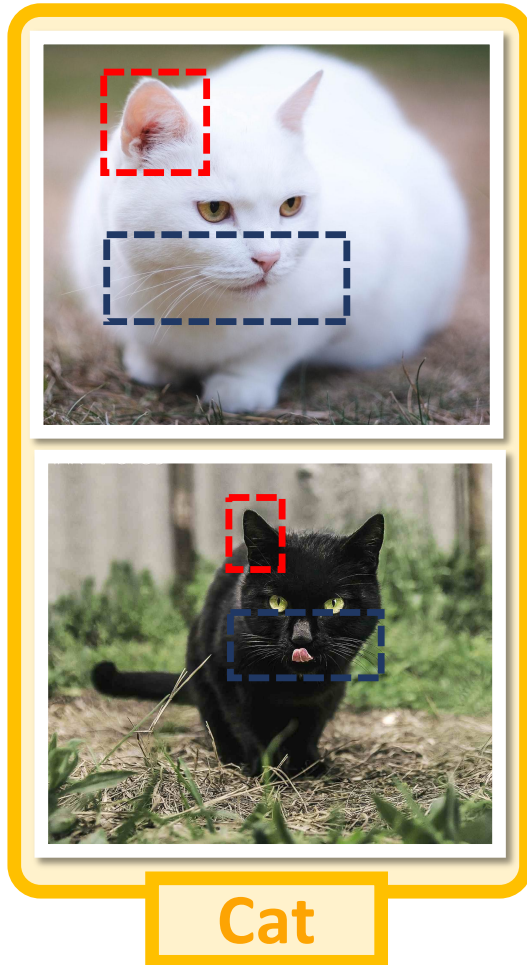


?

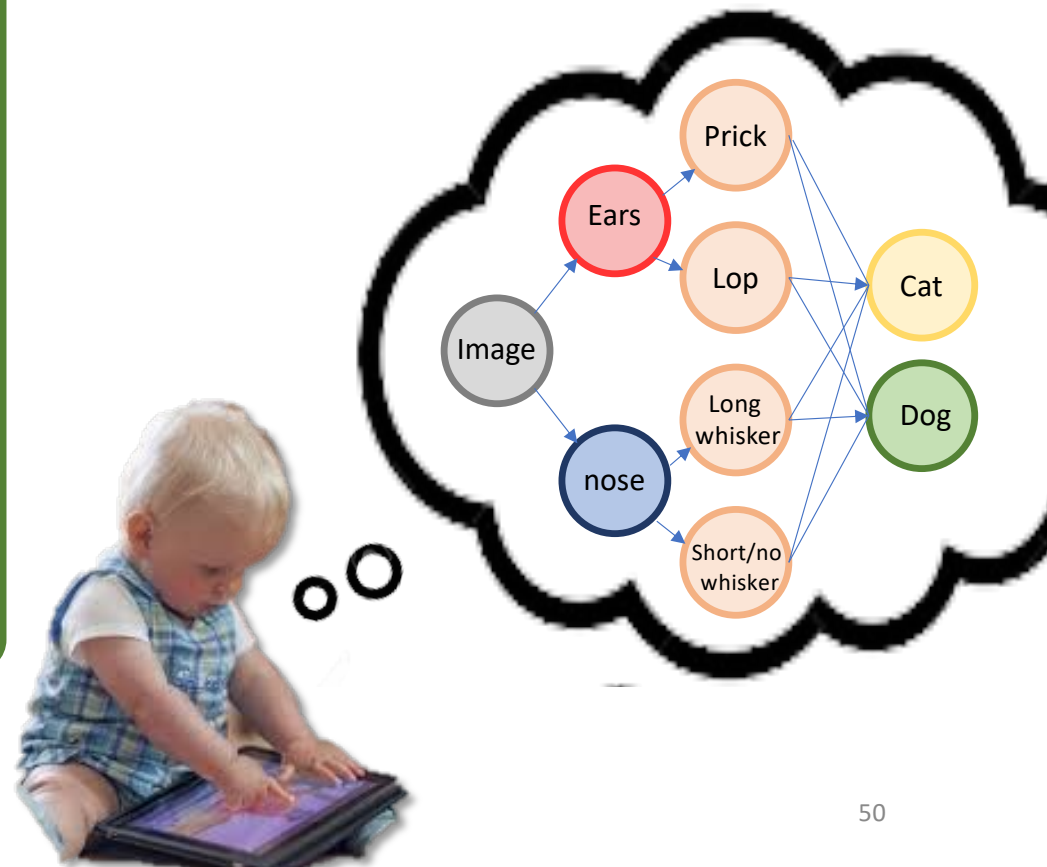


?

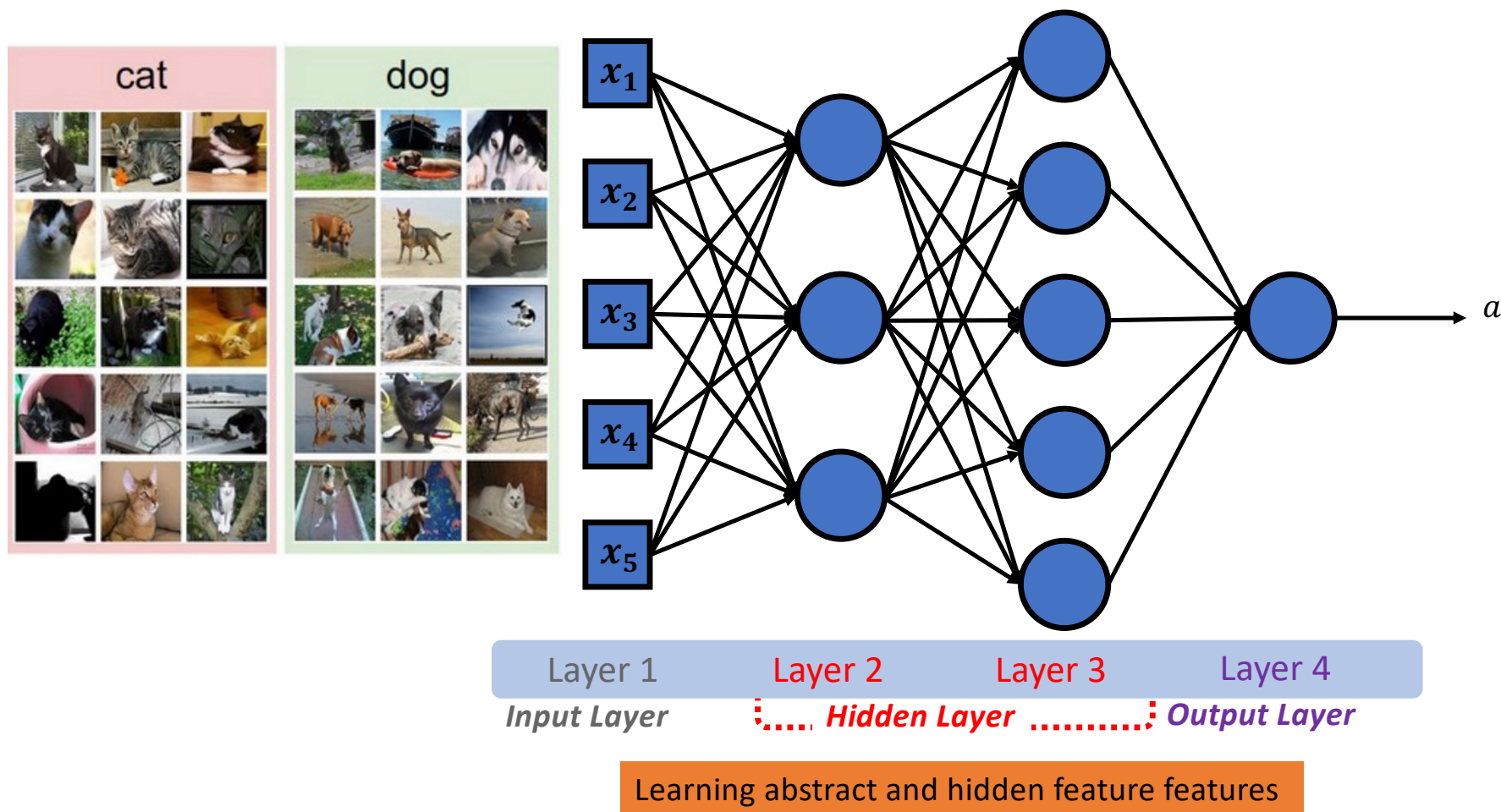
Case 3: Learning



Color → Cannot Classify them
Learning abstract and hidden feature features:
Ears and nose(whisker)



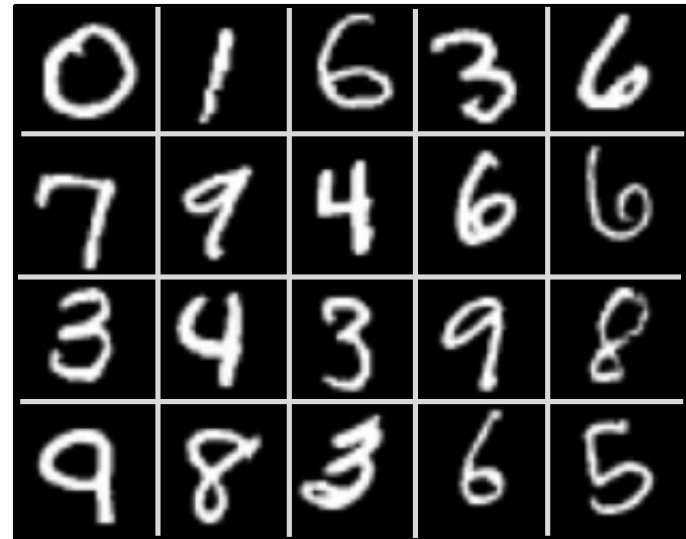
Do the same thing with machines we show them pictures tell them what's in them and hope they figure out all the important features by themselves

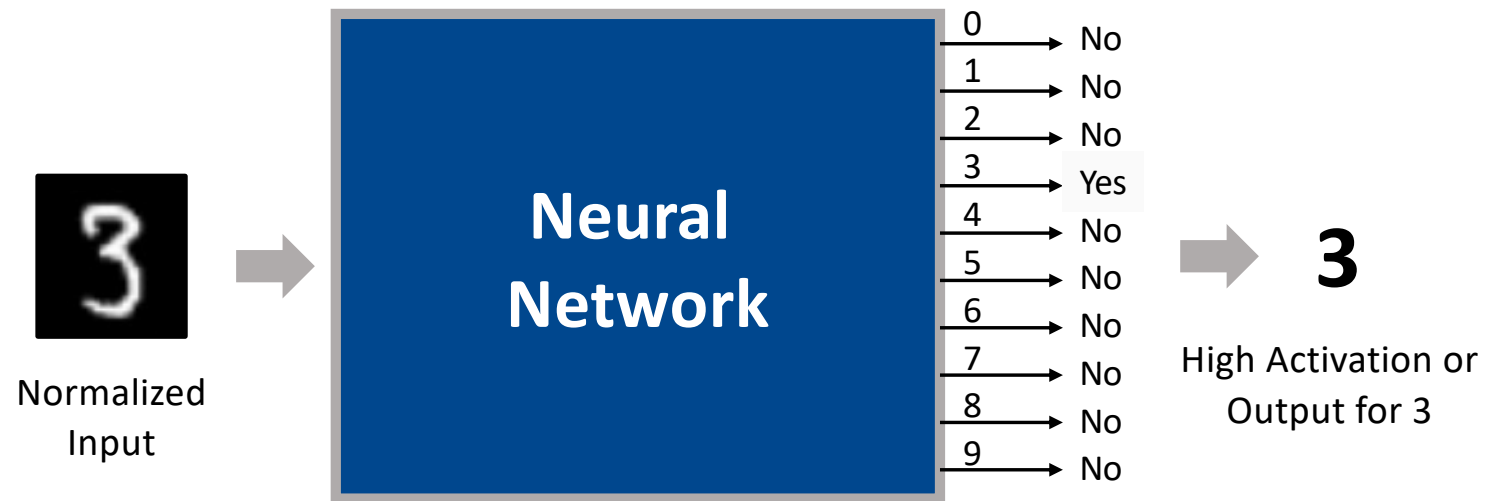


Recognizing Handwritten Decimal Digitals

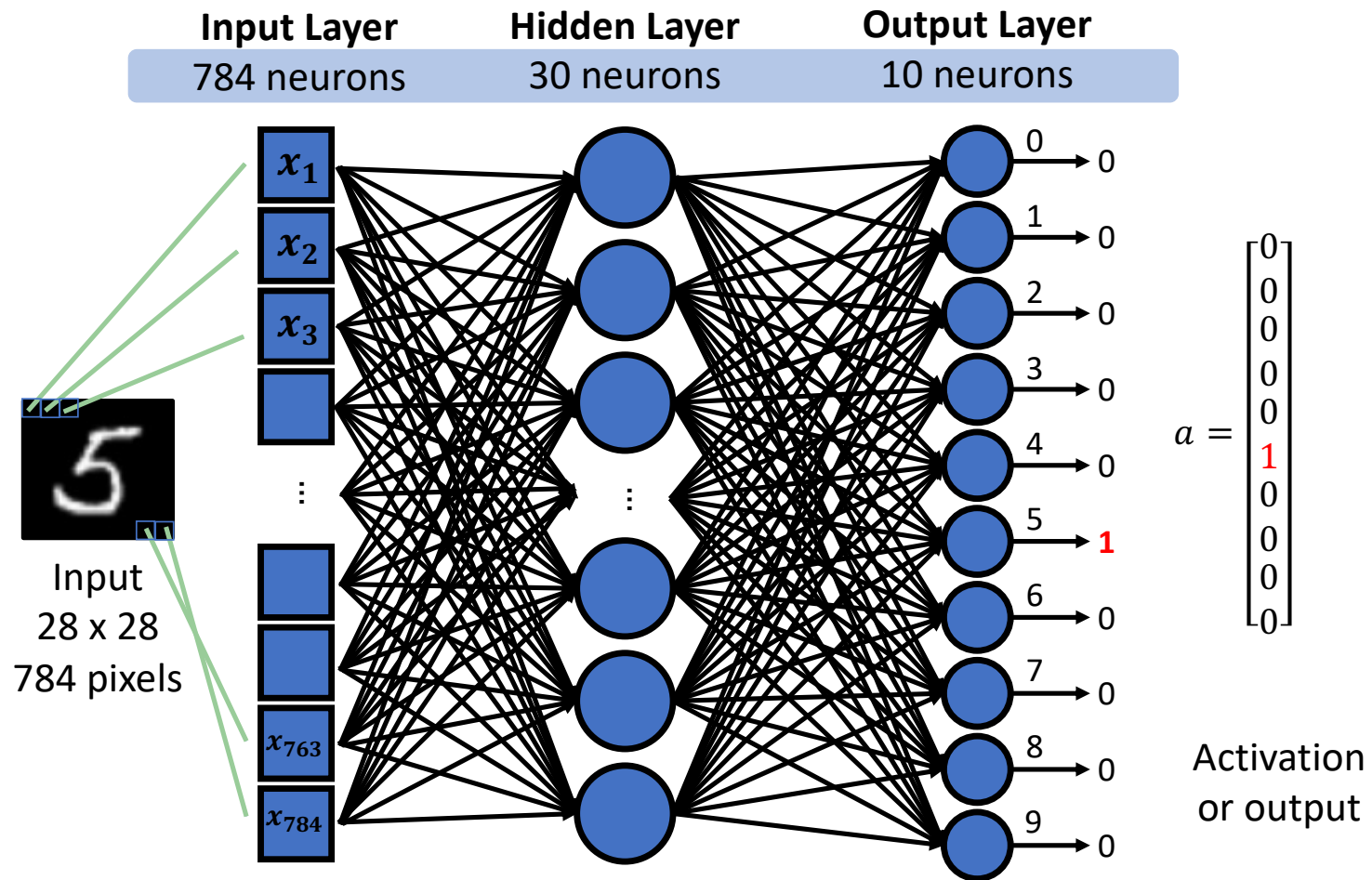
- MNIST Database:

- Handwritten digits database.
- widely used for testing the performance of various networks.
- each handwritten digits are binarized, and then rescaled into an image that is roughly 20 by 20 pixels.
- Image is centered in a larger image, which is about 28 by 28 pixels in size.

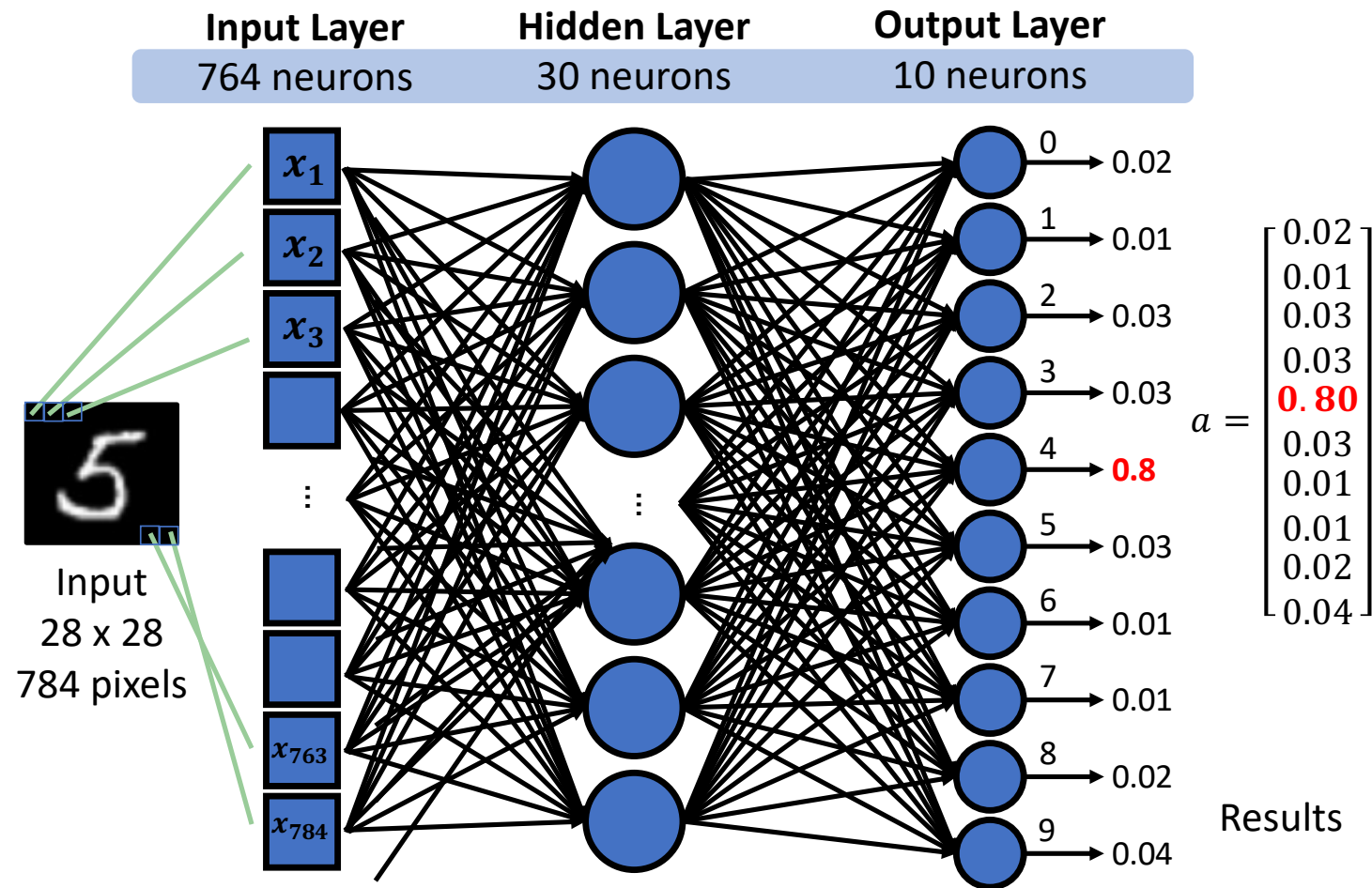




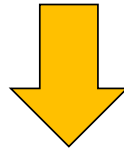
Character Recognition Network



What will happen if we simply randomize the weights?



How to adjust the weights in such a way
that our outputs end up approaching the
desired results?



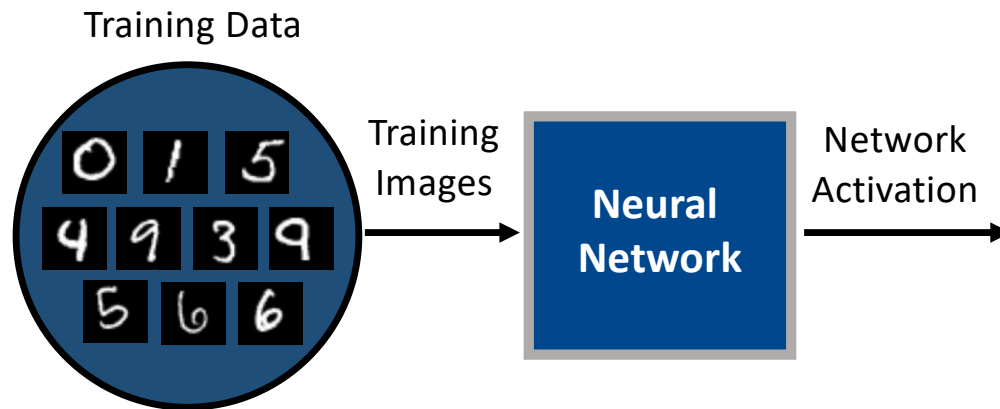
Training

Training Data

- Using training data with known desired activations.
- MINIST Dataset:** 60,000 Sample Training Data

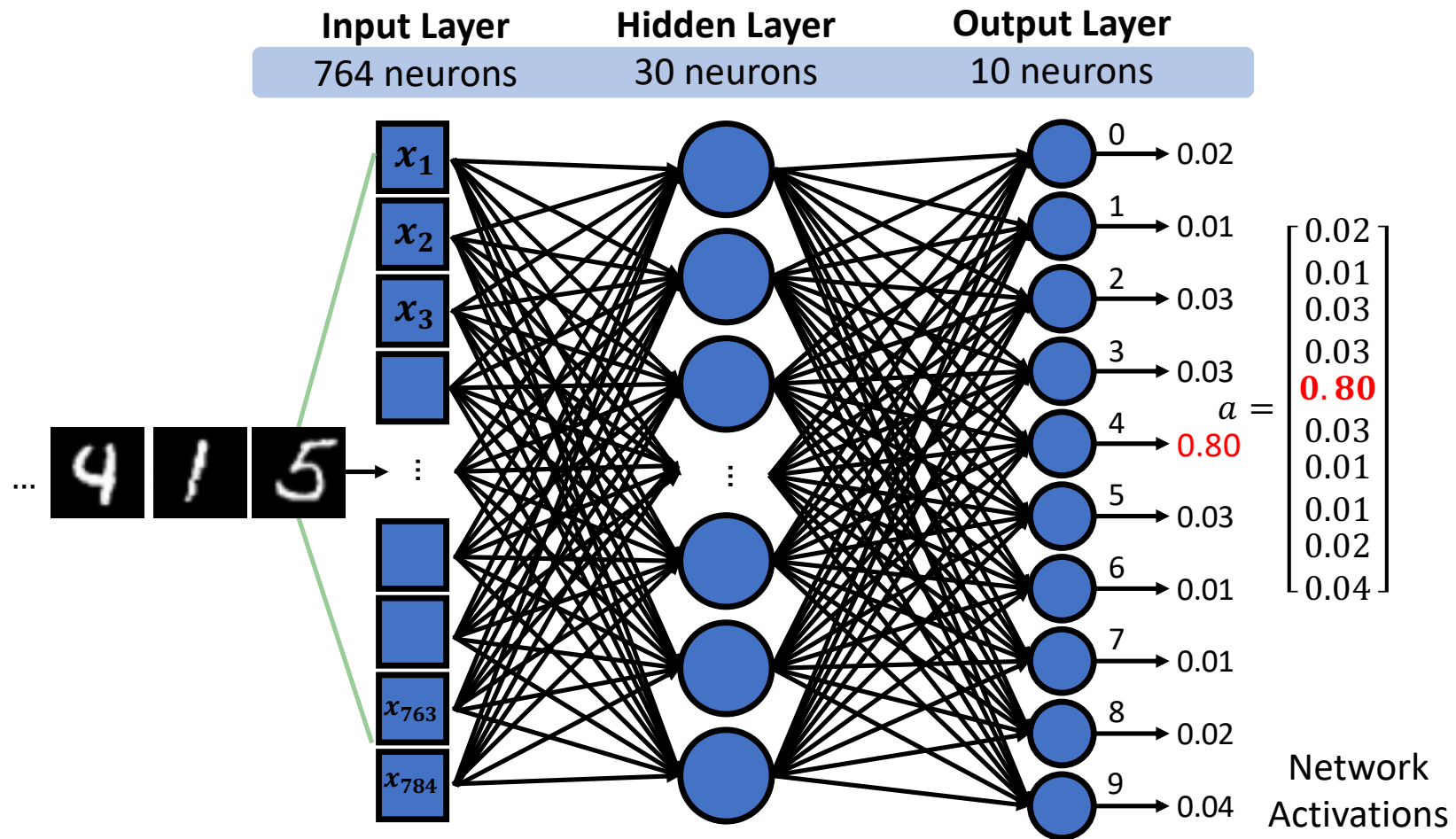
Training Image x										
Label	0	4	1	5	9	3	9	5	6	6
Desired Activation $\hat{a}(x)$	$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

Training Process

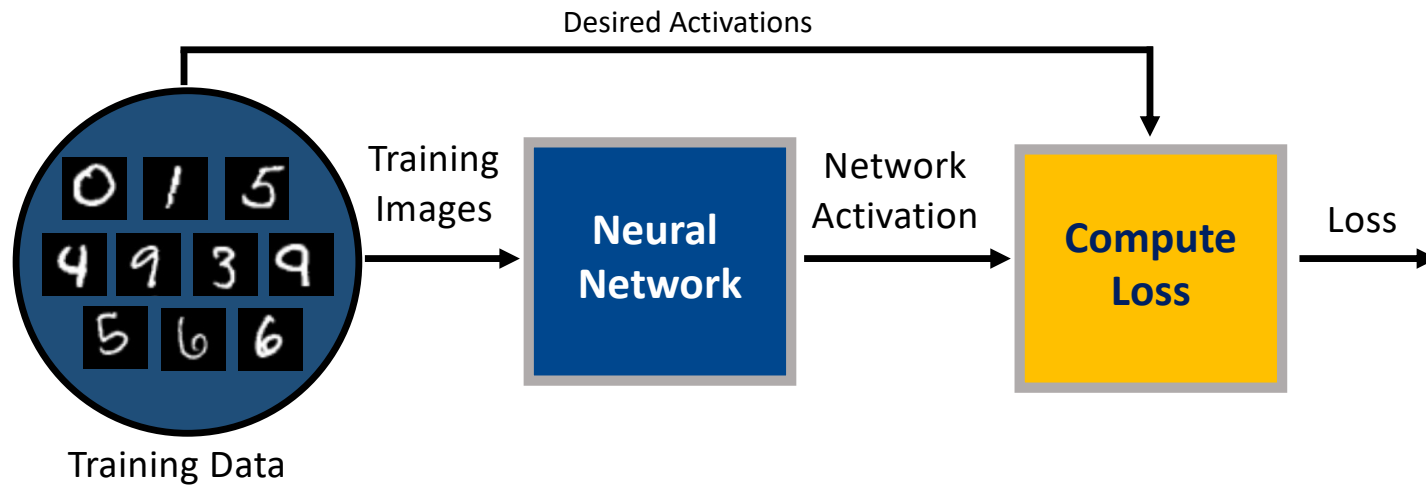


- 1) Randomly initialize Weights
- 2) Determine network activation/result for each training image

Compute Activations: Feedforward



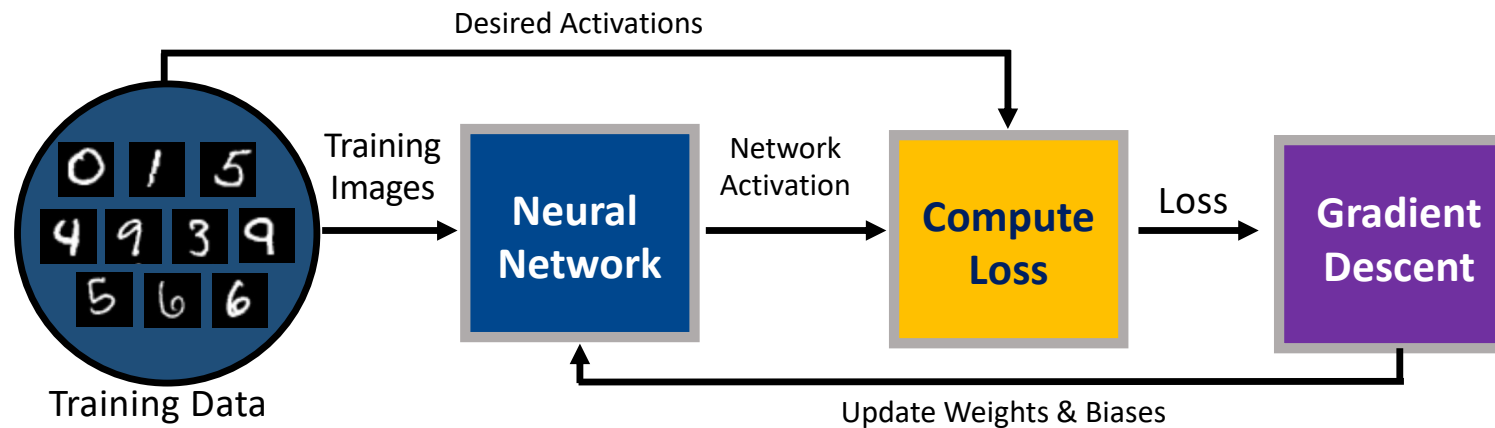
Training Process



- 1) Randomly initialize Weights and Biases
- 2) Determine network activation for each training image
- 3) Compute **Loss** for the entire training image

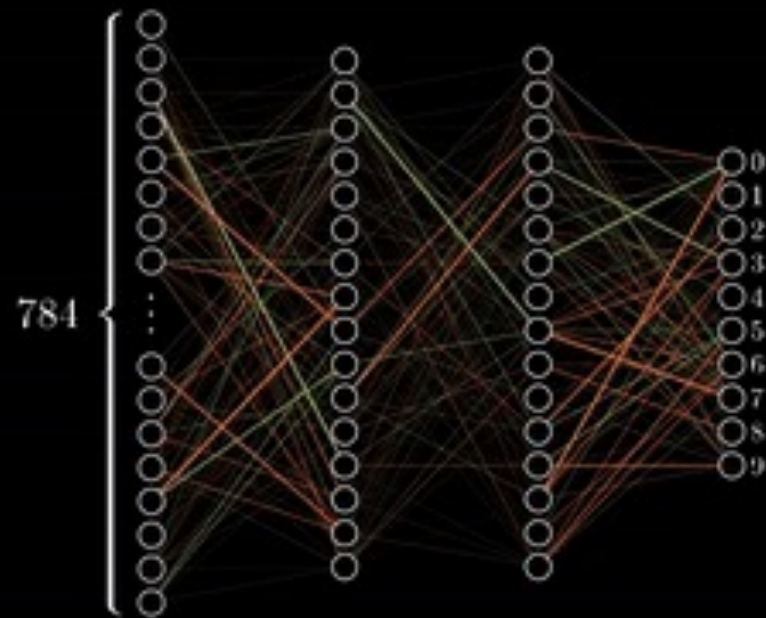
The weaker the performance of the network, the larger the Loss.

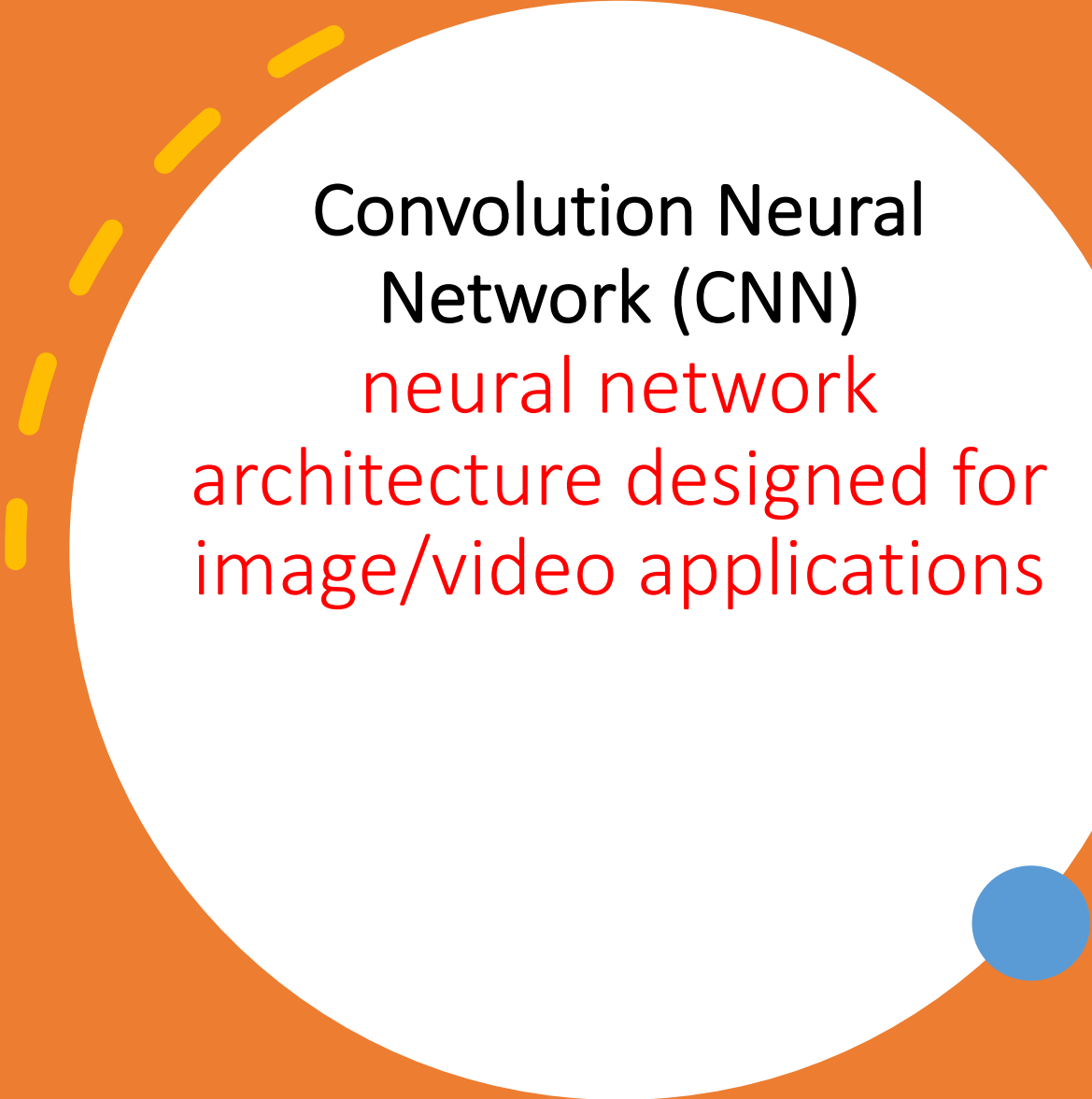
Training Process



- 1) Randomly initialize Weights and Biases
- 2) Determine network activation for each training image
- 3) Compute Loss for the entire training image
- 4) Update Weights using **Gradient Descent**
- 5) **Epoch**: Repeat Steps 2 - 4 until **Loss reduces to an Acceptable Level**

Training in
progress...





Convolution Neural
Network (CNN)
neural network
architecture designed for
image/video applications

Image Classification: Character Recognition

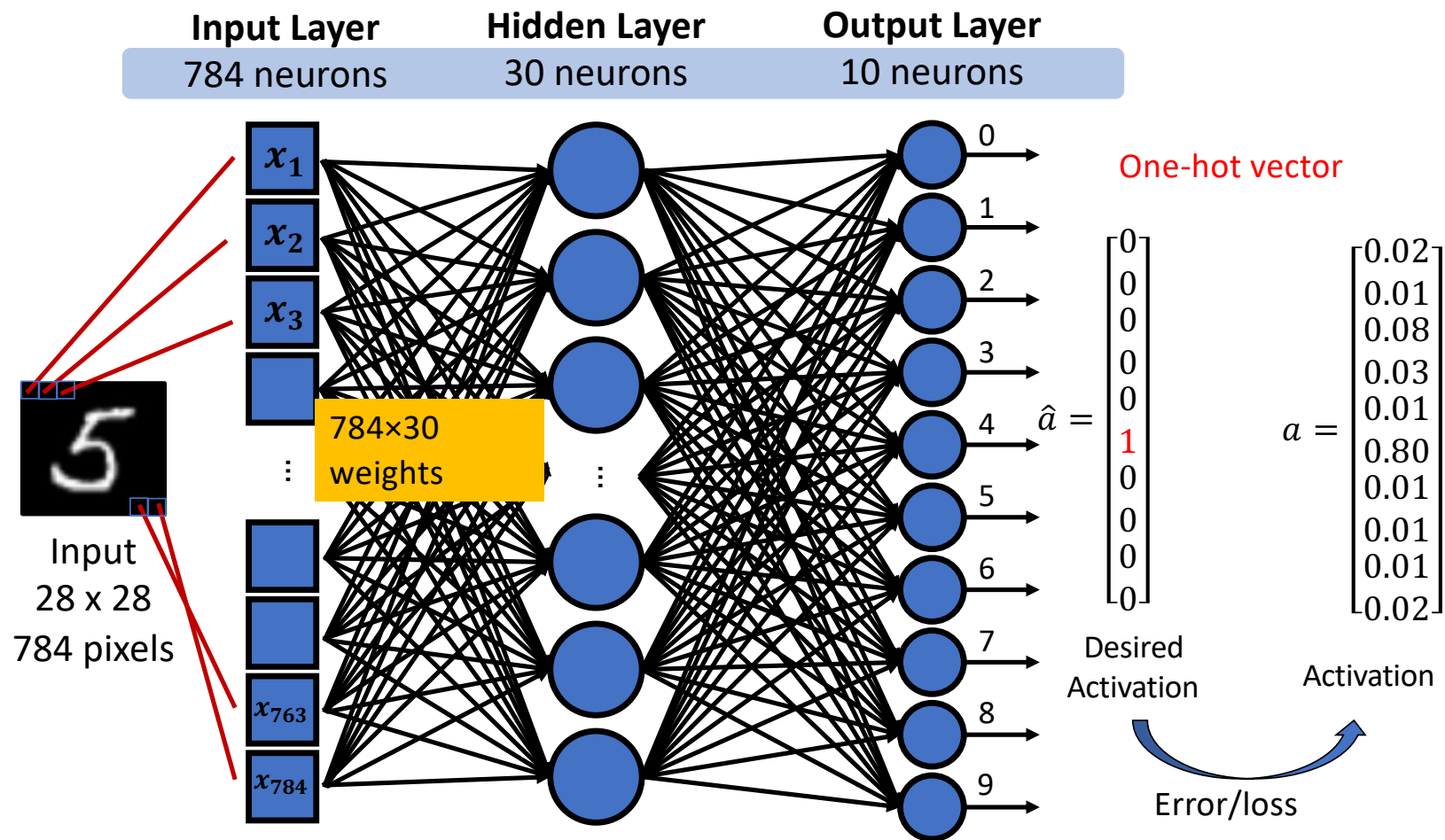
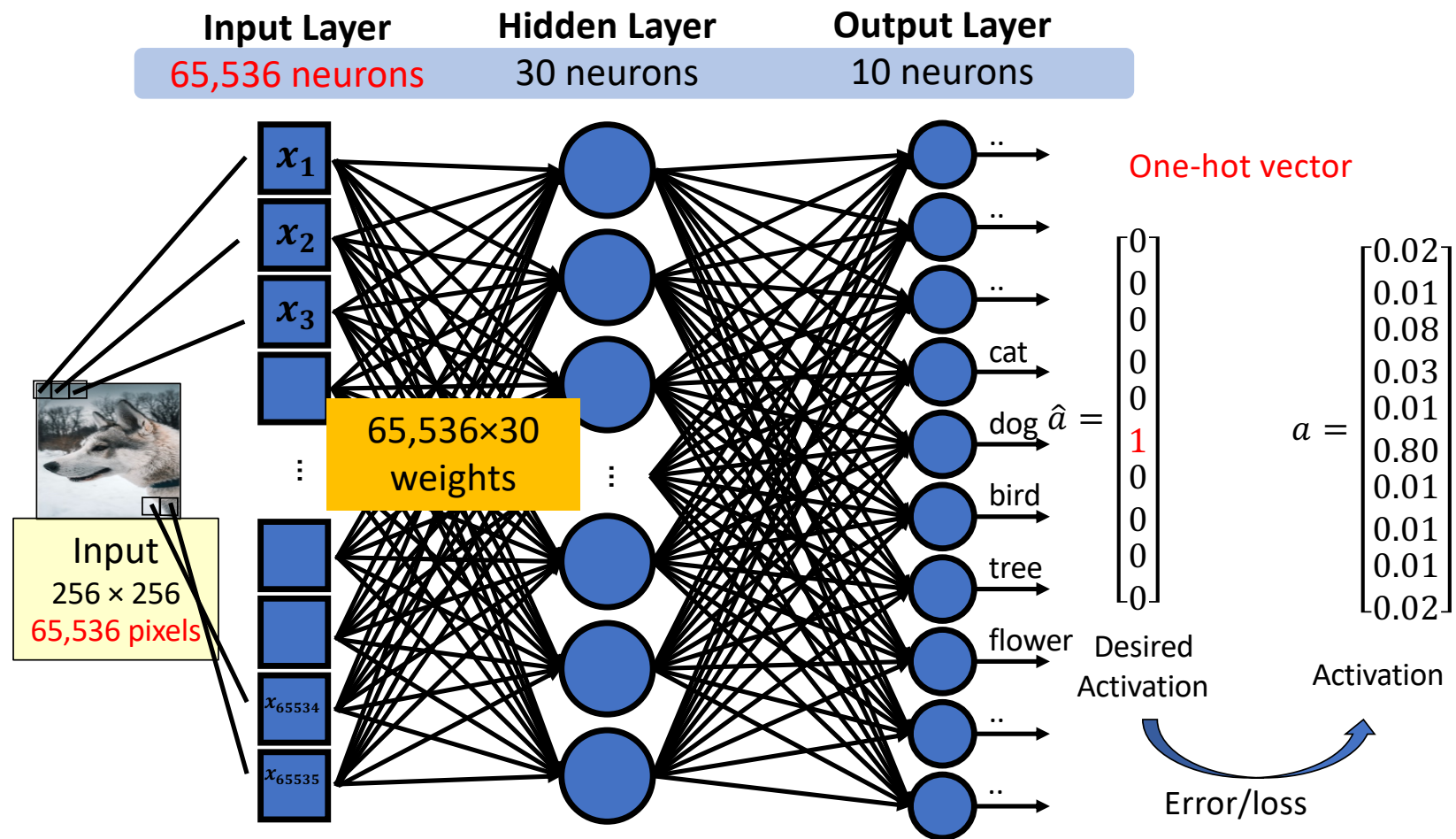
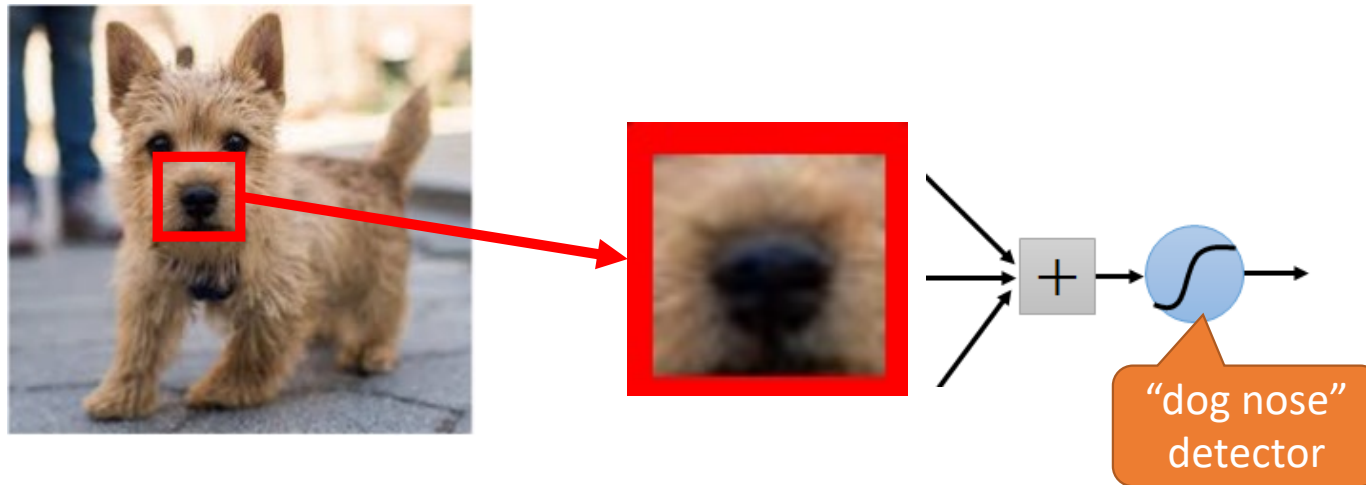


Image Object Classification



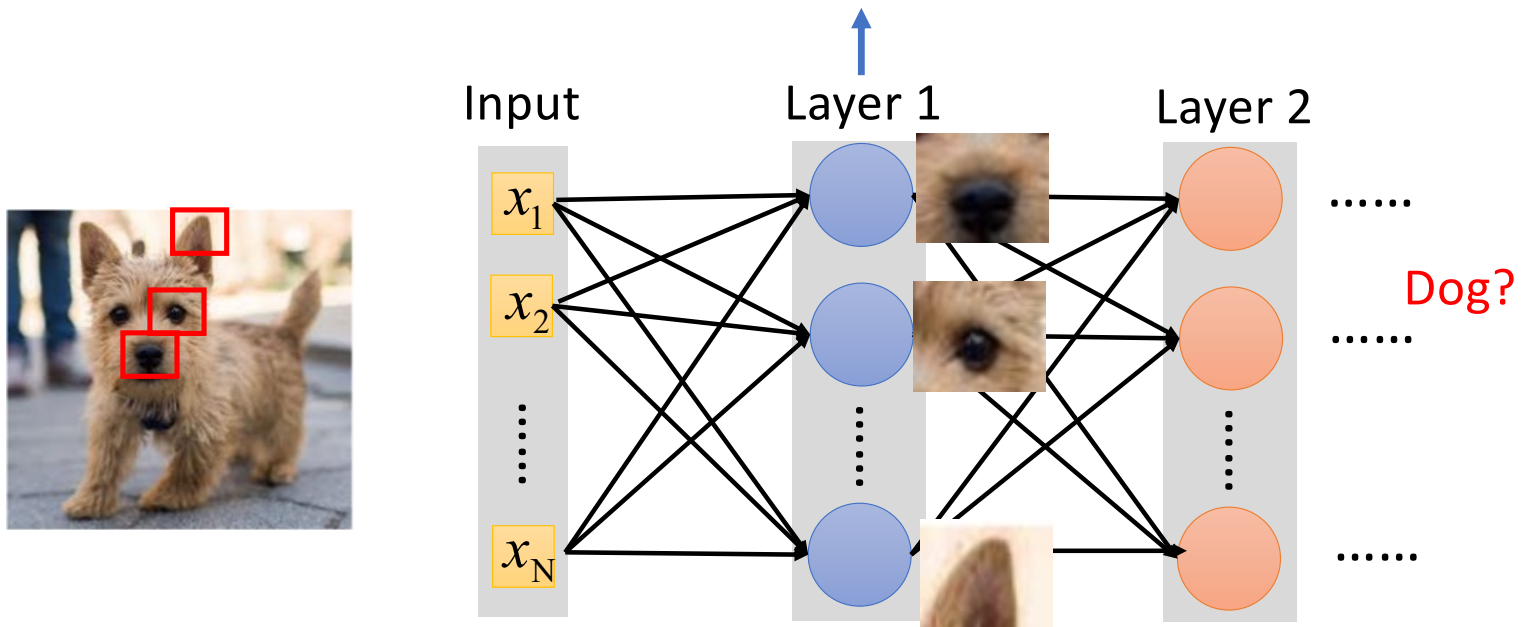
Observation: Some patterns are smaller than the whole image.

A neuron does not have to see the whole image to discover the pattern.



Objective: connecting to small region with less parameters

Recognizing some critical patterns for dogs



Inspiration: It is likely that human identify dogs in a similar way ...

CNN - Summary

- Neurons is to judge whether there are **any patterns appearing**
 - we don't necessarily need that much neurons to take the whole picture as input.
 - They only need to take a small part of the picture as input, which is enough for them to detect whether there exists some critical patterns.